



Carl-Zeiss-Gymnasium Jena
Fachbereich INFORMATIK

<http://www.carl-zeiss-gymnasium.de>

Programmieren mit **Object Pascal**

Mirko König

Kai Rodeck

Mario Schönitz

Diese Unterlagen dienen der Informatik-Ausbildung der Spezialklassen am Carl-Zeiss-Gymnasium Jena der Klassenstufen 9 und 10. Sie sind in erster Linie Arbeitsanleitung zur selbstständigen Tätigkeit und bieten darüberhinaus Übungsmaterialien.

Dieses Dokument wurde mit $\text{\LaTeX}$ gesetzt.

©2004-2010 M. König, K. Rodeck, M. Schönitz

Jena, 11. November 2010

Inhaltsverzeichnis

1. Einföhrung	1
1.1. Charakterisierung	1
1.2. Compiler – Übersetzer	1
1.3. Was ist <i>Lazarus</i> ?	2
2. Arbeiten mit Lazarus	3
2.1. Projekt erstellen	3
2.2. Das erste Programm	3
2.3. Der Objektinspektor	4
2.4. Einige Komponenten	4
2.5. Übungen	5
3. Grafik 1	7
3.1. Zeichnen auf dem Formular	7
3.2. TImage und TPaintBox	8
4. Variablen und Datentypen	11
4.1. Object-Pascal-Programm	11
4.2. Variablen	11
4.3. Datentypen	12
4.4. Übungen	16
5. Algorithmen	19
5.1. Algorithmusbegriff I	19
5.2. Lineare Strukturen	19
5.3. Verzweigende Strukturen	20
5.4. Strukturen zur Wiederholung	23
5.5. Übungen	27
5.6. Algorithmusbegriff II	28
6. Datenstrukturen	29
6.1. Zeichenketten	29
6.2. Ein- und mehrdimensionale Reihungen	32
6.3. Verbund	36
7. Arbeiten mit Dateien	39
7.1. Textdateien	39
7.2. Komplexe Aufgaben	42
I. Regula Falsi	45
II. Datentypen	47
III. Zeichenketten	49
IV. Arten von Programmiersprachen	51
V. Übungsaufgaben	53
V.1. Verzweigte Strukturen	53
V.2. Wiederholende Strukturen	53
V.3. Datenstrukturen	54

1

Lerneinheit Eins

Einführung

Pascal wurde an der ETH Zürich unter Federführung von Niklaus Wirth in erster Linie zu Ausbildungszwecken entwickelt. Die erfolgreiche Vermarktung dieser Sprache gelang der Firma Borland unter dem Namen »Turbo Pascal«. Die Weiterentwicklung der Programmierumgebung von Turbo Pascal führte zur Entwicklung eines RAD-Tools, eines »Rapid Application Development Tools«. ⇒ *Delphi*

Anders Hejlsberg¹ war damals maßgeblich an der Entscheidung von Borland, Delphi zu entwickeln, beteiligt.

Delphi ist ein komponentenorientiertes Programmiersystem, welches auf der Weiterentwicklung von Pascal zu *Object Pascal* basiert. Leider konzentrierte sich die Firma Borland ihre Entwicklung auf Windows. Deshalb entstand der Wunsch, einen Pascaldialekt zu entwickeln, der nicht nur kompatibel zu *Object Pascal*, sondern auch noch portabel auf verschiedenen Plattformen ist, **Free Pascal**.

Die für **Free Pascal** entwickelte IDE² ist **Lazarus**.

1.1. Charakterisierung

Object Pascal als Programmiersprache ist eigentlich eine Neuentwicklung, welche sich eng an Turbo Pascal anlehnt. ⇒ Es ist *keine* Erweiterung desselben!

Tabelle 1.1.: Programmierparadigmen

	Strukturiertheit	Modularität	Objektorientiertheit
Pascal (Wirth)	X		
Turbo Pascal	X	X	(X)
Object Pascal	X	X	X
C	X		
C++	X	X	X
Java	X	X	X
C#	X	X	X

1.2. Compiler – Übersetzer

Man unterscheidet Compiler und Interpreter bei der Übersetzung von Algorithmen auf einem Computer.

Interpreter liest einen Algorithmus Zeile für Zeile und führt diese sofort aus.

Compiler analysiert erst den gesamten Algorithmus auf Korrektheit, übersetzt in Maschinencode und erstellt dann ein ausführbares Programm³.

Pascal ist eine Compiler-Sprache!

¹Seit 1996 Chefentwickler bei Microsoft und verantwortlich für C# und DotNET

²Integrated Development Environment

³Das ausführbare Programm wird i. E. durch *Linker* realisiert

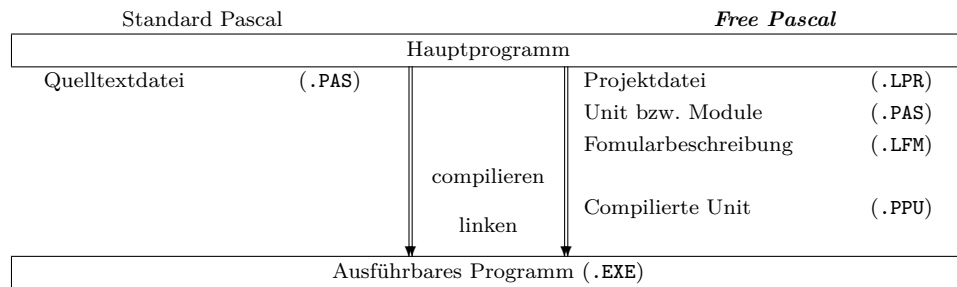


Abbildung 1.1.: Standard Pascal vs. Free Pascal

Somit existieren die folgenden Dateien im Projektordner:

Projekt1.lpr	–	
Unit1.pas	–	
Unit1.lfm	–	
→ compilieren	–	(Übersetzen in Maschinen[Objekt]code)
Unit1.ppu	–	
Project1.lrs	–	
Project1.compiled	–	
→ linken	–	(Binden des Objektcodes mit Bibliotheken)
Project1.exe	–	

1.3. Was ist Lazarus?

Lazarus (RAD-Tool) hilft, Programme für die jeweilige Plattform ⁴ entwickeln. Das bedeutet, es werden mittels spezieller Bibliotheken Fenster, Menüs, Buttons und ähnliches als Komponenten bereitgestellt.

Moderne Windowsmanager arbeiten **ereignisorientiert**. Grundelement ist eine *Ereignisschleife*.

Ein Fenster (Formular) ist somit ein dem Windowsmanager untergeordnetes Objekt, welches auf Maus-, Tastatur- oder andere besondere Ereignisse wartet, diese wiederum an seine untergeordneten bzw. eingebetteten Komponenten verteilt und auf deren Antwort wartet.

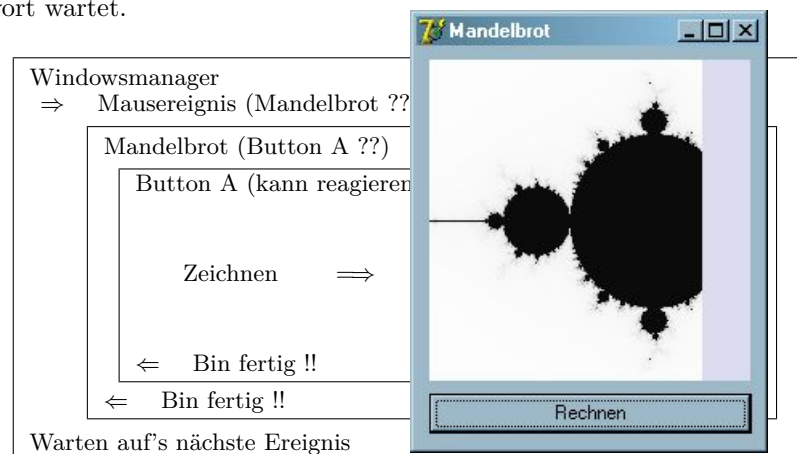


Abbildung 1.2.: Ereignisse

⁴Windows, Linux oder Macintosh

2

Lerneinheit Zwei

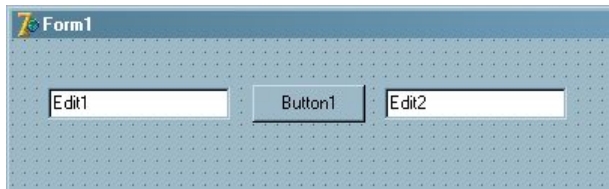
Arbeiten mit Lazarus

2.1. Projekt erstellen

1. Menü : Datei → Neu... → Projekt → Application
und !!! **sofort als Projekt in einem *neuen* Ordner speichern !!!**
2. Menü → Projekt → Projekt speichern unter...

2.2. Das erste Programm

Erstelle ein neues Projekt und stelle das Formular wie dargestellt zusammen!



Diese kleine Programm rechnet die Geschwindigkeiten der Einheit $\frac{m}{s}$ in $\frac{km}{h}$ um.
Doppelklicke auf den **Button1** und vervollständige mit dem folgenden Quelltext.

Die Zahlen vor jeder Zeile sind nicht einzugeben, sie dienen nur der besseren Orientierung!

```
1 procedure TForm1.Button1Click(Sender: TObject);
2 var geschw1,
3     geschw2 : real;           // Kommazahlen
4 begin
5     geschw1:=StrToFloat(Edit1.Text); // Eingabe
6     geschw2:=geschw1 * 3.6;       // Berechnung
7     Edit2.Text:=FloatToStr(geschw2) // Ausgabe
8 end;
```

Vorgehen:

1. Menü: Alles speichern
2. Menü: Start → Schnelles Kompilieren
3. Menü: Start → Start
4. Eingabe einer Zahl in das linke Editfeld - Klicke auf *Button1*!

Erläuterungen:

- (1) Name der Methode, welche auf das Ereignis *Mausklick* reagieren soll. Diese Zeile wird automatisch von der IDE generiert, indem in der Formularansicht doppelt auf die Komponente (hier Button) geklickt wird.
- (2, 3) Deklaration von Variablen. In diesen können Fließkommazahlen gespeichert werden.
- (4, 8) Markierung für den Anfang und das Ende der Definition der Methode. Zwischen diesen Begrenzern stehen die Anweisungen, welche ausgeführt werden sollen.
- (5) Der Inhalt des *Editfeldes1* wird in eine Zahl umgewandelt (*StrToFloat()*) und diese dann in der Variablen *geschw1* zur weiteren Verwendung gespeichert. Der Variablen *geschw1* wird ein Wert zugewiesen (*:= .. Ergibtanweisung*).
- (6) Der Ausdruck *geschw1*3.6* wird ausgewertet und der Variablen *geschw2* als Wert zugewiesen.
- (7) Der Wert der Variablen *geschw2* wird in eine Zeichenfolge umgewandelt (*FloatToStr()*) und als Wert (Inhalt) des Editfeldes2 definiert.

2.3. Der Objektinspektor

Der Objektinspektor verwaltet alle im Formular verwendeten Komponenten sowie deren Eigenschaften. Diese Eigenschaften lassen sich schon im Vorfeld der Programmerstellung ändern.

So lässt sich zum Beispiel die Beschriftung eines Button ändern¹. Dazu ändern wir die Eigenschaft »Caption« und schreiben »Berechne« in das entsprechende Feld. Aufpassen, im Fenster »Objekt-Hierarchie« den Eintrag **Button1** anklicken.

Ändere die Eigenschaft »Text« der Editfelder und lösche den Eintrag »Edit..«.

Ein Programm sollte einen entsprechenden Namen tragen, insbesondere ist der Titel des Formulars zu ändern (Eigenschaft »Caption«).

2.4. Einige Komponenten

Button	Beschreibung	Wichtige Eigenschaften
	<i>Label</i> : Einfache Beschriftungen bzw. Texte auf dem Formular.	Caption, Font, Name
	<i>Memo</i> : Eingabe- und Ausgabefeld für mehrzeilige Texte.	Font, Lines, Name, Lines.Add
	<i>Panel</i> : Hervorgehobene Textausgabe auf dem Formular, zum Beispiel Überschriften etc.	Caption, Color, Font, Name
	<i>MaskEdit</i> : Eingabefeld für vordefinierte Texte (z. B. Datum, PLZ, Zeitangaben etc.).	Text, <i>EditMask</i> , Font, Name

Eine wichtige Eigenschaft aller Komponenten ist *Name*! Mit dieser Eigenschaft lassen sich Bezeichnungen der Komponenten in selbsterklärende Namen² ändern!

z. B. Edit1 → EdtEingabe bzw.
 Edit2 → EdtAusgabe

Lazarus stellt eine Vielzahl von visuellen Komponenten zu Verfügung. Die wichtigsten bzw. am häufigsten verwendeten Komponenten befinden sich in den Registern »Standard« und »Additional«. Den Namen der Komponenten nennt Lazarus, wenn der Mauszeiger längere Zeit auf dem entsprechenden Feld verbleibt. Drückt man F1 (Hilfe), während man die linke Maustaste auf dem Button gedrückt hält, öffnet sich das entsprechende Hilfe-Fenster.

Ändere die Oberfläche unseres ersten Programms. Beschrifte Ein- und Ausgabefeld und füge eine Überschrift ein. Gib dem Formular einen bezeichnenden Namen.

Ändere die dazu notwendigen Eigenschaften im Objektinspektor, wähle dazu im Fenster »Objekt-Hierarchie« die entsprechende Komponente aus.

Erweitere unser Programm um eine weitere Schaltfläche zum Beenden des Programms. Die Anweisung für diesen Button lautet:

```
Close();
```

¹Dies sollte man *immer* tun

²Besonders, wenn viele gleiche Komponenten verwendet werden

2.5. Übungen

1. Entwirf und implementiere ein Programm in *Object Pascal*, das die Inhalte zweier Editfelder vertauscht.
Verwende nacheinander die Datentypen `real` (für Kommazahlen) und `string` (für Zeichenketten).
2. Entwirf und implementiere ein Programm in *Object Pascal* zur Berechnung am Rechteck!

Hinweis:

Orientiere dich an dem unten gezeigten Formular sowie am Quelltext unseres ersten Programms.

Deklariere die Variablen `seite_a`, `seite_b`, `flaeche`, `umfang`, `diagonale` als `real`.

Eingabe → Editfelder - Seite a und Seite b

Verarbeitung → `flaeche := seite_a * seite_b;`
`umfang := 2 * (seite_a + seite_b);`
`diagonale := sqrt(sqr(seite_a) + sqr(seite_b));`

Ausgabe → Editfelder - Fläche, Umfang, Diagonale

Abbildung 2.1.: Formblatt - Berechnung

3. Entwirf und implementiere ein Programm, das aus der gegebenen Seitenlänge eines Würfels sein Volumen, seinen Oberflächeninhalt sowie die Länge der Raumdiagonalen berechnet!
4. Entwirf und implementiere ein Programm zur Temperatur-Umrechnung von Grad Celsius in Grad Fahrenheit und in Kelvin.

Hinweis:

`fahrenheit = celsius * 1,8 + 32` `kelvin = celsius + 273`

5. Implementiere ein Programm, das die Tilgungsrate eines Kredites berechnen kann.

Soll eine Schuld S in n Jahren bei einem Zinssatz $p\%$ p.a. durch regelmäßige Ratenzahlungen jeweils am Jahresende getilgt werden, so beträgt die Rate:

$$R = \frac{S \cdot q^n (q - 1)}{q^n - 1}, \text{ mit } q = 1 + \frac{p}{100}$$

Hinweis:

Mit der Funktion `Power()` können beliebige Potenzen berechnet werden. Informiere dich in der Hilfe von Lazarus über die Verwendung dieser Funktion.

3

Lerneinheit Drei

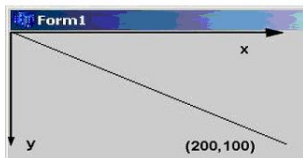
Grafik 1

3.1. Zeichnen auf dem Formular

Das Formular, selbst eine Komponente, stellt eine Eigenschaft bereit, die gewährleistet, dass sich alle untergeordneten Komponenten auf dem Formular zeichnen können:

Canvas (Zeichenfläche)

Es handelt sich hierbei um die rechteckige Fläche einer Pixelgrafik. Das zugrunde liegende Koordinatensystem hat seinen Ursprung in der linken oberen Ecke des Fensters.



Diese Linie wurde mit der Anweisung (Programmzeile)

```
Form1.Canvas.LineTo(200,100);
```

 erzeugt.

Abbildung 3.1.: Koordinatensystem

Ausgewählte Grafik-Befehle: (beziehen sich immer auf ein Canvas)

- `MoveTo(x,y);`
- `LineTo(x1,y1);`
- `Rectangle(x1,y1,x2,y2);`
- `Ellipse(x1,y1,x2,y2);`
- `Pen.Color := clRed;`
- `Brush.Color := clWhite;`
- **Aufgabe 1 D**
efiniere Buttons mit den Aufschriften Haus und Sonne. Definiere die `onClick`-Ereignisse der Buttons, so dass jeweils ein Haus und eine Sonne gezeichnet wird.
- **Aufgabe 2 I**
n Eingabefeldern vom Typ Edit sollen Ganzzahlen eingegeben werden, die in Balkendiagrammen dargestellt werden.



Die Ausrichtung der Balken kann über die linke Position der Editfelder (↗ *Eigenschaften*) erfolgen.

```
Rectangle(Edit?.Left, ...);
```

Teste, was passiert, wenn die Werte der Editfelder die Höhe (*Eigenschaft:Height*) des Formulars überschreiten!

Teste und ändere, wenn die Werte der Editfelder geändert werden und die Balken neu gezeichnet werden sollen.

Notwendige Typumwandlungen zum Benutzen von Text-Komponenten für Berechnungen (Edit, Memo, ..):

```
// Ganzzahl ↔ Zeichenkette
IntToStr();   StrToInt();
// Kommazahl ↔ Zeichenkette
FloatToStr(); StrToFloat();
```

3.2. TImage und TPaintBox

Außer dem Formular gibt es die Komponenten *Image* und *PaintBox*, welche über die Eigenschaft einer Zeichenfläche (Canvas) verfügen. Obwohl beide Komponenten im Wesentlichen über die gleichen Eigenschaften und Methoden verfügen, gibt es dennoch wesentliche Unterschiede.

1. *onPaint-Methode* (in der Lage sein, sich neu zu zeichnen)

Erweitere das Programm aus Aufgabe 2!

Zeichne die Balken nacheinander jeweils in ein Image und eine PaintBox!

Komponentenleiste: „Additional“



bzw.

Füge einen Button ein:

```
1 procedure TForm.Button2Click(Sender: TObject);
2 begin
3     Application.MessageBox('Achtung!', 'PaintBox_??_Image',
4                             MB_OK);
5 end;
```

Was stellst du fest?

2. Die Fähigkeit, sich selbst zu speichern.

Im Gegensatz zu *TImage* stellt *TPaintBox* keine Eigenschaft *Picture* zur Verfügung, sie kann also keine Bilder speichern. Eine PaintBox ist aber dafür bei seiner Darstellung um einiges schneller.

TImage eignet sich also zum Darstellen und Speichern von Grafiken, *TPaintBox* mehr »nur« zum Zeichnen.

```
Image1.Picture.LoadFromFile(Dateiname);
Image1.Picture.SaveToFile(Dateiname);
```

● Aufgabe 3 D

as Sierpinski – Dreieck

Erstelle ein neues Projekt, mit einem Image sowie drei Buttons!

Ändere im Objektinspektor die Namen der drei Buttons in `BtnZeichnen`, `BtnSpeichern` und `BtnBeenden`.

Erstelle durch Doppelklick auf den jeweiligen Button die zugehörige Ereignisprozedur.

```
1 // Ereignis – Zeichnen der Figur
2 procedure TForm1.BtnZeichnenClick(Sender: TObject);
3   var i,j : integer;
4   begin
5     for i:=0 to 255 do
6       for j:=0 to 255 do
7         if i and j = j then
8           Image1.Canvas.Pixels[i,j]:=clBlue;      //Setze Punkt
9         end;
10 // Ereignis – Speichern der Figur
11 procedure TForm1.BtnSpeichernClick(Sender: TObject);
12   begin
13     Image1.Picture.SaveToFile('sierpinski.png');    //Speichern
14   end;
15 // Ereignis – Beenden des Programmes
16 procedure TForm1.BtnBeendenClick(Sender: TObject);
17   begin
18     Close();                                     // Programm beenden
19   end;
```


4

Lerneinheit Vier

Variablen und Datentypen

4.1. Object-Pascal-Programm

Ein Object-Pascal-Programm ist eine Folge von Anweisungen, mit deren Hilfe ein zugrunde liegendes Problem in endlich vielen Schritten gelöst wird. Zu diesem Zweck wird ein Datenverarbeitungsprozess initiiert, der aus zwei logischen Komponenten besteht:

1. Daten und Vereinbarungen zu deren Organisation: Daten sind konkrete Werte, mit denen ein Programm arbeitet. Diese Werte können entweder von Beginn an fest vorgegeben sein – wir bezeichnen diese Werte als Konstanten – oder sie besitzen einen Anfangswert, welcher während des Datenverarbeitungsprozesses verändert werden kann – diese Werte werden als Variablen bezeichnet.
2. Anweisungen zur Verarbeitung der Daten: Es sind Anweisungen zur Steuerung des Programmablaufs wie zum Beispiel die ein-, zwei- oder mehrseitige Auswahl, die Wiederholung mit voran- oder nachgestelltem Test oder die gezählte Wiederholung, aber auch Anweisungen, welche die Verarbeitung der Daten regeln. Dies sind unter anderem Ein- und Ausgabeanweisungen, aber auch arithmetische und logische Operationen.

Daten und Anweisungen werden im Speicher eines Computers getrennt voneinander verwaltet. Dazu wird der Speicher in zwei Bereiche aufgeteilt: Es sind das Datensegment zur Speicherung der Daten und das Codesegment zur Speicherung der Anweisungen. Wird ein Programm ausgeführt, unterteilt das Betriebssystem den für das Programm reservierten Speicher automatisch in Daten- und Codesegment, lädt das Programm in das Codesegment und die zur Verarbeitung notwendigen Daten in das Datensegment. Da vor der Programmausführung nicht immer vorhergesagt werden kann, welche Daten verarbeitet werden – häufig werden sie erst vom Anwender zur Laufzeit des Programms festgelegt – gibt es zwei weitere Speicherbereiche, die im Gegensatz zum Datensegment dynamisch mit Daten belegt werden können: Es sind der Heap und der Stack.

4.2. Variablen

Eine Variable ist ein Container für einen Wert eines bestimmten Typs. Eine Variable besteht aus ihrem Namen und einem Wert, der durch diesen Namen repräsentiert wird. Jede Deklaration einer Variablen hat eine Reservierung eines eindeutig definierten Speicherbereiches zur Folge, in dem der Wert der Variablen abgelegt wird.

Der Wert einer Variablen ist nach deren Deklaration undefiniert und muss daher immer explizit definiert werden. Der Zugriff auf den in der Variablen gespeicherten Wert erfolgt über den Namen der Variablen. In Object Pascal werden Variablen durch das Schlüsselwort **var** deklariert. Die Deklaration einer Variablen muss immer vor dem Anweisungsteil eines Programms beziehungsweise Unterprogramms erfolgen.

```
1 // Beispiel zum Umgang mit Variablen.
2 procedure TForm1.Button1Click(Sender: TObject);
3 // Deklaration der Variablen balance und payment.
4 var balance, payment : Real;
5 begin
6     // Die Variable balance wird initialisiert.
7     balance := 2000.00;
8     // Die Variable payment wird initialisiert.
9     payment := 500.50;
```

```

10 // Der Wert der Variablen balance wird um den Wert der
11 // Variablen payment erhöht. Die Variable balance hält
12 // anschließend den Wert 2500.50 – der Wert der Variablen
13 // payment bleibt unverändert.
14 balance := balance + payment;
15 end;

```

4.3. Datentypen

In Abhängigkeit davon, welche Werte in einer Variablen gespeichert werden sollen, müssen unterschiedlich große Speicherbereiche reserviert werden. Zu diesem Zweck besitzt jede Variable einen Datentyp, der festlegt, welche Werte die Variable aufnehmen kann und welche Operationen auf dem Variablenwert möglich sind. In *Object Pascal* (*Free Pascal*) stehen unter anderem die folgenden Datentypen zur Verfügung:

Datentypen zur Speicherung ganzer Zahlen (Integer-Typen)

In *Free Pascal* beschreiben alle Integer-Typen jeweils eine echte Teilmenge aus der Menge der ganzen Zahlen.

Tabelle 4.1.: *Integer-Typen*

Name	Minimum	Maximum	Operationen	Speicherbedarf in Byte
Byte	0	$2^8 - 1$	+	1
Word	0	$2^{16} - 1$	–	2
LongWord	0	$2^{32} - 1$	*	4
QWord	0	$2^{64} - 1$	div und mod	8
ShortInt	-2^7	$2^7 - 1$	=	1
SmallInt	-2^{15}	$2^{15} - 1$	>	2
LongInt	-2^{31}	$2^{31} - 1$	<	4
Int64	-2^{63}	$2^{63} - 1$	>=	8
Cardinal	0	$2^{32} - 1$	<=	4
Integer	-2^{15}	$2^{15} - 1$	<>	2

```

1 procedure TForm1.Button1Click(Sender: TObject);
2 var dividend, divisor, quotient, rest: Word;
3 begin
4   dividend := 97;
5   divisor := 15;
6   // Ganzzahlige Division. quotient hält danach den Wert 6.
7   quotient := dividend div divisor;
8   // Rest der ganzzahligen Division. rest hält danach den Wert
9   // 7.
10  rest := dividend mod divisor;
11 end;

```

Die Typen `Cardinal` und `Integer` sind generische Typen.

Datentypen zur Speicherung rationaler Zahlen (Float-Typen)

In Object Pascal beschreiben alle Float-Typen jeweils eine echte Teilmenge aus der Menge der rationalen Zahlen.

Tabelle 4.2.: *Real-Typen*

Name	Minimum	Maximum	Operationen	Speicherbedarf in Byte
Single	$1.5 \cdot 10^{-45}$	$3.4 \cdot 10^{38}$	+, -, *	4
Double	$5.0 \cdot 10^{-324}$	$1.7 \cdot 10^{308}$		8
Extended	$1.9 \cdot 10^{-4932}$	$1.1 \cdot 10^{4932}$	=, >, <	10
Real	$5.0 \cdot 10^{-324}$	$1.7 \cdot 10^{308}$	>=, <=, <>	8

```

1 procedure TForm1.Button1Click(Sender: TObject);
2 var dividend, divisor, quotient: Single;
3 begin
4   dividend := 355.0;
5   divisor := 113.0;
6   // Gleitkommadivision. quotient hält danach den Wert
   // 3.14159292.
7   quotient := dividend / divisor;
8 end;
```

Der Typ `Real` ist ein generischer Typ.

Datentypen zur Speicherung einzelner Zeichen (Character-Typen)

In Object Pascal ist ein Character-Typ ein Typ, mit dessen Hilfe eine endliche Menge von einzelnen Zeichen abgebildet wird. In Abhängigkeit vom konkreten Typ sind dies alle Zeichen aus dem ASCII beziehungsweise alle Zeichen aus dem UNICODE.

Tabelle 4.3.: *Character-Typen*

Name	Minimum	Maximum	Operationen	Speicherbedarf (Byte)
Char	ASCII 0	ASCII 127	CHR	1
WideChar	UNICODE 0	UNICODE 65535	ORD	2

Die Zeichen ASCII 0 bis ASCII 127 stimmen mit den Zeichen Unicode 0 bis Unicode 127 überein. Daher ist es möglich, ein Zeichen mit dem Unicode 65 einer Variablen vom Typ `Char` zuzuweisen. Einer Variable vom Typ `Char` kann kein Zeichen mit einem Unicode größer als 127 zugewiesen werden.

```

1 procedure TForm1.Button1Click(Sender: TObject);
2 var plus, minus, plusminus: Char;
3 begin
4   plus := '+';
5   minus := chr(45);
6   // Laufzeitfehler.
7   plusminus := chr(177);
8 end;
```

Um auf einzelne Zeichen mit einem Unicode größer 127 zuzugreifen, sind spezielle Umwandlungsroutinen notwendig.


```

1 procedure TForm1.Button1Click(Sender: TObject);
2   var dollar, euro: WideChar;
3       unicode: Word;
4   begin
5     dollar := '$';
6     // Die Variable dollar enthält die
7     // WideChar-Darstellung des druckbaren Zeichens '$'.
8     euro := UTF8Decode('\EUR')[1];
9     // Die Variable euro enthält die
10    // WideChar-Darstellung des druckbaren Zeichens '\EUR'.
11    unicode := ord(euro);
12    // Die Variable unicode enthält den Wert 8364. Dies ist
13    // der Unicode des Zeichens '\EUR' in dezimaler Schreibweise.
14  end;

```

Datentypen zur Speicherung einer Folge von Zeichen (String-Typen)

Ein String beziehungsweise eine Zeichenkette ist eine endliche Folge von Zeichen. Eine Zeichenkette, die keine Zeichen enthält wird als Leer-String beziehungsweise leere Zeichenkette bezeichnet. Hinweis: In Object Pascal ist 1234 eine Zahl und '1234' eine Zeichenkette, die aus den Zeichen '1', '2', '3' und '4' besteht. Beide Notationen sind streng voneinander zu unterscheiden.

Tabelle 4.4.: *String-Typen*

Name	Maximallänge in Zeichen	Speicherbedarf
ShortString	255	2 Byte bis zu 256 Byte
AnsiString	speicherabhängig	1 Byte pro Zeichen
WideString	speicherabhängig	2 Byte pro Zeichen
String	speicherabhängig	1 Byte pro Zeichen

```

1 procedure TForm1.Button1Click(Sender: TObject);
2   var plate: AnsiString;
3   begin
4     plate := 'J┐-┐FK┐63';
5   end;

```

Eine Variable vom Typ ShortString kann maximal 255 Zeichen vom Typ Char aufnehmen. Wird einer Variablen vom Typ ShortString eine Zeichenfolge zugewiesen, die auch Zeichen vom Typ WideChar enthält, verringert sich deren Aufnahmekapazität entsprechend, da ein Zeichen vom Typ WideChar einen Speicherbedarf von 2 Byte besitzt. Der Typ String ist mit dem Typ AnsiString identisch. Wird bei einer Typdeklaration vom Typ String eine maximale Länge festgelegt, so ist der Typ String mit dem Typ ShortString identisch.

Datentypen zur Speicherung von Wahrheitswerten (Boolesche Typen)

Ein boolescher Typ ist ein Typ, mit dessen Hilfe Wahrheitswerte abgebildet werden können. Da die boolesche Logik eine zweiwertige Logik ist, werden nur die beiden Werte true und false unterschieden. Hinweis: In Object Pascal ist true ein Wahrheitswert und 'true' eine Zeichenkette, die aus den Zeichen 't', 'r', 'u' und 'e' besteht. Beide Notationen sind streng voneinander zu unterscheiden.

```

1 procedure TForm1.Button1Click(Sender: TObject);

```

```

2 var comparison: Boolean;
3   a, b : Byte;
4 begin
5   a := 12;
6   b := -12;
7   // Die Variable comparison erhält den Wert true.
8   comparison := (a < 7) OR (b < 0);
9 end;

```

Tabelle 4.5.: Boolesche Typen

Name	Minimum	Maximum	Operationen	Speicherbedarf in Byte
Boolean	false	true	AND, OR, NOT, XOR	1

```

1 procedure TForm1.Button1Click(Sender: TObject);
2 var comparison: Boolean;
3   a, b : Byte;
4 begin
5   a := 12;
6   b := -12;
7   // Die Variable comparison hält den Wert true.
8   comparison := (a < 7) OR (b < 0);
9 end;

```

Typkompatibilität

Eine Variable kann nur Werte des Datentyps aufnehmen, dessen Wertebereich eine Teilmenge des Datentyps ist, von dem sie deklariert wurde.

```

1 procedure TForm1.Button1Click(Sender: TObject);
2 var balance : Word;
3   payment : Single;
4 begin
5   // Compilerfehler.
6   balance := 2000.00;
7   // Initialisierung der Variablen.
8   balance := 2000;
9   payment := 500.50;
10  // Compilerfehler.
11  balance := balance + payment;
12  // Initialisierung der Variablen.
13  payment := 200;
14  // Compilerfehler.
15  balance := balance + payment;
16 end;

```

Object Pascal stellt Funktionen zur Verfügung, mit deren Hilfe Werte vom Typ `String` in Werte eines ordinalen oder eines `Float`-Typs und umgekehrt überführt werden können.

Jede Variable muss durch einen Namen, den so genannten Bezeichner eindeutig definiert sein. Für die Namensgebung gelten die folgenden, in der erweiterten Backus-Naur-Form dargestellten Regeln:

```

identifizier = letter | "_" { letter | digit | "_" }.
letter = "A" | "B" | ... | "Z" | "a" | "b" | ... | "z".
digit = "0" | "1" | ... | "9".

```

Tabelle 4.6.: Typkonvertierung

Funktion	Beschreibung	Beispiel
StrToInt	Konvertiert eine Zeichenkette in einen Integer-Wert.	<code>intValue := StrToInt('971');</code>
StrToFloat	Konvertiert eine Zeichenkette in einen Float-Wert.	<code>floatValue := StrToFloat('3.14');</code>
StrToBool	Konvertiert eine Zeichenkette in einen booleschen Wert.	<code>boolValue := StrToBool('true');</code>
IntToStr	Konvertiert einen Integer-Wert in eine Zeichenkette.	<code>strValue := IntToStr(971);</code>
FloatToStr	Konvertiert einen Float-Wert in eine Zeichenkette.	<code>strValue := FloatToStr(3.14);</code>
BoolToStr	Konvertiert einen booleschen Wert in eine Zeichenkette.	<code>strValue := BoolToStr(true);</code>

Object Pascal unterscheidet bei Bezeichnern nicht zwischen Groß- und Kleinschreibung. Bezeichner können eine beliebige Länge haben, jedoch werden in Object Pascal nur die ersten 127 Zeichen unterschieden. Reservierte Wörter dürfen nicht als Bezeichner gewählt werden.

4.4. Übungen

- Überprüfen Sie, ob es sich bei den folgenden Ausdrücken um gültige Bezeichner handelt:

```

1 _1234
2 height
3 1234_
4 länge
5 _String
6 account-balance
7 _width_
8 car_plate
9 shortINT

```

- Welche der folgenden Anweisungen verursacht einen Compilerfehler und warum?

```

1 procedure TForm1.BerechnenClick(Sender: TObject);
2 var a,b,c,d      : ShortInt;
3   x,y,z          : Single;
4   k,l            : Extended;
5   gelb, grün     : Boolean;
6   m, n           : String;
7 begin
8   a := 10;
9   b := 5;

```

```

10  n := 1;
11  k := 16734,789;
12  x := 2 * (a + b);
13  y := a / b;
14  c := x - 2 * y;
15  l := y - a mod b;
16  m := "2008 - Jahr der Mathematik";
17  gelb := b > c;
18  d := b / a;
19  z := 2 * l;
20  end;

```

3. Welches Wort verbirgt sich hinter dem Unicode U+0141, U+0075, U+006B, U+0061, U+0073, U+0069, U+0065, U+0077, U+0069, U+0063, U+007A?
4. Begründe, warum in Lazarus einer Variablen vom Typ `Char` nicht der Wert 'ä' zugewiesen werden kann.
5. Welchen Wert besitzt die Variable `p` nach der Hintereinanderausführung folgender Wertzuweisungen:

```

1  a := 2;
2  p := a;
3  p := p * p;
4  p := a * p;
5  p := p * p;

```

Welchen Wert hält `p` für `a = 3`? Welche Funktion berechnet die Anweisungsfolge für einen beliebiges `a`?

6. Die Variablen `a` und `b` sind mit den Werten 2 und 7 belegt. Es wird die Anweisungsfolge

```

1  b := a + b;
2  a := b - a;
3  b := b - a;

```

ausgeführt. Welche Werte halten die Variablen nach der Ausführung der Anweisungsfolge? Lässt sich das Resultat auch ohne die Verwendung von Rechenzeichen erzielen? Teste, ob das dargestellte Verfahren allgemeingültig ist.

7. Banken benötigen zur Zinsberechnung die Anzahl der Tage zwischen zwei Tagesdaten. Ein Tagesdatum ist durch die Angabe von drei natürlichen Zahlen eindeutig definiert: Es sind jeweils eine Zahl für Tag, Monat und Jahr. Jeder Monat hat 30 Tage, jedes Jahr 360 Tage. Entwickle ein Verfahren, mit dessen Hilfe die Anzahl der Tage zwischen zwei gegebenen Tagesdaten ermittelt werden kann.
8. Implementiere eine Anwendung in Object Pascal, mit deren Hilfe die Anzahl der Tage zwischen zwei gegebenen Tagesdaten ermittelt werden kann.
9. Ein Sportverein plant einen Ausflug mit einem öffentlichen Verkehrsmittel. Ein Tarifkilometer kostet 20 Cent, bei Gruppenreisen wird jeder sechsten Person eine Freifahrt gewährt. Entwickle einen Algorithmus, mit dessen Hilfe die Fahrtkosten pro Person ermittelt werden können: Wir nehmen an: Die Gesamtkosten werden auf alle Teilnehmer gleichmäßig umgelegt. Setze deinen Algorithmus anschließend als Object-Pascal-Anwendung um.
10. Stadtwerke gestalten die Abrechnung für die Lieferung von elektrischer Energie in der Regel wie folgt: Der Zählerstand wird einmal pro Jahr abgelesen. Der Kunde zahlt monatlich einen gewissen Abschlagsbetrag, wobei am Jahresende der Nachzahlungs- beziehungsweise Erstattungsbetrag ermittelt wird. Die Abrechnungsformulare werden elektronisch erstellt. Dazu gibt der Sachbearbeiter die folgenden Daten ein: Alter und neuer Zählerstand in Kilowattstunden, bisherige Abschlagsrate in Euro, jährliche Grundgebühr des Anschlusses in Euro, Preis pro

verbrauchter Kilowattstunde in Euro und aktueller Umsatzsteuersatz in Prozent. Das Abrechnungsformular gibt einen Überblick über den Absolutverbrauch im zurückliegenden Abrechnungszeitraum, den durchschnittlichen Verbrauch pro Monat, die Energiekosten für den zurückliegenden Abrechnungszeitraum, den darin enthaltenen Umsatzsteuerbetrag, eine mögliche Erstattung beziehungsweise Nachzahlung sowie den Abschlag für den neuen Abrechnungszeitraum. Entwirf und implementiere eine Anwendung in Object Pascal, mit deren Hilfe auf der Grundlage notwendiger Eingabegrößen ein Abrechnungsformular erstellt werden kann. Verwende zur Ausgabe eine Instanz der Komponente Memo.

11. Ein Dreieck sei durch die Koordinaten seiner Eckpunkte gegeben. Entwirf und implementiere eine Anwendung in Object Pascal, mit deren Hilfe festgestellt werden kann, ob ein weiterer gegebener Punkt im Inneren, auf dem Rand oder außerhalb des Dreiecks liegt.

5

Lerneinheit Fünf

Algorithmen

5.1. Algorithmusbegriff I

Ein Algorithmus¹ ist eine festgelegte Folge von eindeutigen, ausführbaren Anweisungen zur Lösung einer Klasse von Problemen.

- Auf jede Anweisung folgt genau eine weitere, oder der Algorithmus ist beendet.
- Jede Anweisung berücksichtigt die von ihren Vorgängern veränderten Eigenschaften.
- Gleiche Eingaben in den Algorithmus erzeugen die gleichen Ausgaben.

Strukturen		
<i>lineare</i>	<i>verzweigende</i>	<i>wiederholende</i>
Sequenz	Einseitige Auswahl	Zählschleife
Wertzuweisung	Zweiseitige Auswahl	... mit Eintrittsbedingung
Unterprogramm	Mehrfach-Auswahl	... mit Austrittsbedingung

5.2. Lineare Strukturen

Lineare Strukturen haben wir bereits in den letzten Stunden und Programmen kennen gelernt. Hier noch mal eine kurze Zusammenfassung.

Sequenz

Ein Folge von Anweisungen oder Befehlen an den Computer bezeichnet man als Sequenz. Wird diese Folge in BEGIN und END eingeschlossen, wird wieder von einer Anweisung oder einem Anweisungsblock gesprochen.

Wertzuweisung

Die wichtigste Anweisung ist die Wertzuweisung. Dabei handelt es sich um eine Wertbelegung von Variablen. Variablen sind Objekte, die neben einem Namen (Bezeichner) auch einen Typ und einen Wert besitzen.

Variable (Name, Typ, Wert)

Die einfachste Anweisung, einer Variablen einen Wert zu zuweisen, ist die Ergibtanweisung.

```
geschw2 := geschw1 * 3.6;      oder
Edit1.Text := FloatToStr(geschw2);      oder
Edit2.Text := 'mayer';
```

¹ Die Bezeichnung „Algorithmus“ geht auf den arabischen Kaufmann und Schriftsteller Mohammed ibn Musa al-Chwarismi (9.Jh.) aus Bagdad zurück. Eine seiner vielen Schriften, die „Schrift der Rechenkunst“, enthält Regeln für die vier Grundrechenarten im Dezimalsystem. Diese Regeln können als formale Verarbeitungsvorschriften für Ziffernfolgen aufgefasst werden, mit denen man ohne tieferes mathematisches Verständnis, Rechenoperationen durchführen kann. Bis ins 17.Jh. wurden Anhänger des schriftlichen Rechnens mit Dezimalzahlen Algorithmiker genannt. Nach seinem Buch „Al gabr w'al muqabalah“ hat auch die Algebra ihren Namen erhalten.

Unterprogramm

Es ist eine Folge von Anweisungen, die einen Namen (Bezeichner) bekommen und unter diesem Bezeichner kann dann diese Anweisungsfolge verwendet werden. Zusätzlich ist es möglich, nur für dieses Unterprogramm Variablen zu deklarieren (lokale Variable, nur gültig in diesem Block) sowie ihm Werte zu übergeben (Wertzuweisung mittels Parameter).

```

1 // Ereignismethode OnClick
2 procedure TForm1.Button1Click(Sender: TObject);
3                               // Sender ist formaler Parameter
4 var beispiel:integer;         // lokale Variable
5 begin
6     . . .                    // Hier die Anweisungen
7 end;
```

5.3. Verzweigende Strukturen

Ein- und zweiseitige Auswahl

Auswahlstrukturen dienen dazu, den weiteren Programmablauf festgelegt zu bestimmen. Es existieren zwei grundsätzliche Möglichkeiten, eine Entscheidung zu treffen.

Bedingung ist eine Frage, ob ein Ereignis eingetreten ist. (WAHR oder FALSCH) Es stehen höchstens zwei Wege zu Auswahl.

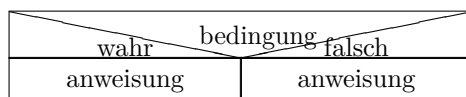
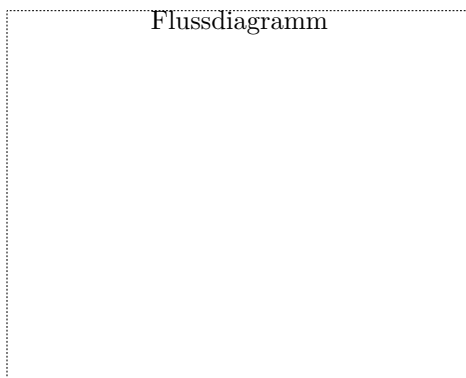
Wertbelegung einer Variablen, entsprechend existieren mehrere Wege für den weiteren Programmablauf.

Zweiseitige Auswahl

```

1 IF bedingung THEN
2     anweisung
3 ELSE
4     anweisung;
```

Flussdiagramm



```

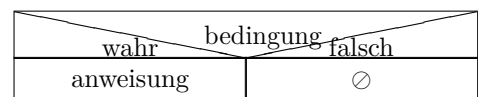
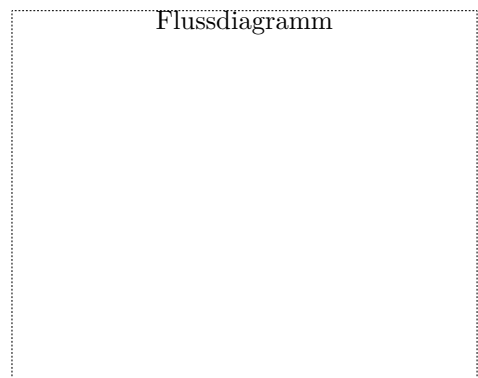
1 ...
2 IF a>0 THEN
3     absolut := a
4 ELSE
5     absolut := -1 * a;
6 ...
```

Einseitige Auswahl

```

1 IF bedingung THEN
2     anweisung;
```

Flussdiagramm



```

1 .
2 .
3 IF a>0 THEN
4     absolut := a;
5 .
6 .
```

In Abhängigkeit einer Bedingung, wird entweder die eine oder die andere Anweisung ausgeführt.

In Abhängigkeit einer Bedingung, wird eine Anweisung zusätzlich ausgeführt.

- Vor einem ELSE darf kein Semikolon stehen!

Merke

- Bedingung ist ein logischer Ausdruck, der einen Wahrheitswert liefert!
- In den Zweigen darf nur eine Anweisung stehen! Lösung - siehe (5.2)!

Beispiel

Zur Entscheidung, ob für quadratische Gleichungen Lösungen existieren, kann die Diskriminante verwendet werden.

Beispiel: Quadratische Gleichungen

Eingabe: a, b, c	
$dis = \frac{b^2 - 4ac}{4a^2}$	
$dis \geq 0$	
ja	nein
Ausgabe: 'Lösungen' 'JA'	Ausgabe: 'Lösungen' 'NEIN'

```

1  var a, b, c , dis : real;
2  begin
3      // notwendige Eingaben
4      . . .
5      // Diskriminante berechnen
6      dis:=(b*b-4*a*c)/(4*a*a);
7      if dis>=0 then
8          EdAusgabe.Text:=
9              'Es_gibt_Lösungen'
10     else
11         EdAusgabe.Text:=
12             'Es_gibt_keine_Lösungen'
13             ;
14     end;

```

Mehrfach - Auswahl

Es gibt immer wieder Probleme, bei deren Lösung es häufig nicht ausreicht, einfach nur zwischen wahr und falsch zu unterscheiden. Diese können mit *IF*-Abfragen nur unzureichend behandelt werden, was zu einer sehr unübersichtlichen Beschreibung des Algorithmus führen kann.

Beispielsweise sind bei der Zuordnung einer verbalen Bewertung und der entsprechenden Zensur sechs Fälle zu unterscheiden.

Die Umsetzung dieses Problems mit Hilfe von *IF*-Anweisungen ist aufwendig (*Schreibarbeit*) und fehleranfällig.

Lösung:

- 1 → sehr gut
- 2 → gut
- 3 → befriedigend
- 4 → ausreichend
- 5 → genügend
- 6 → ungenügend

```

1  text := '';
2  read (note);
3  case note of
4      1: text := 'Sehr_Gut';
5      2: text := 'Gut';
6      3: text := 'Befriedigend';
7      4: text := 'Ausreichend';
8      5: text := 'Genügend';
9      6: text := 'Ungenügend';
10 end;
11 write(text);

```


Übungen

1. Entwirf und implementiere ein Programm, welches mit Hilfe der Lösungsformel die Nullstellen einer quadratischen Funktion in Normalform ermittelt und gegebenenfalls ausgibt!

Zeichne dazu ein Struktogramm!

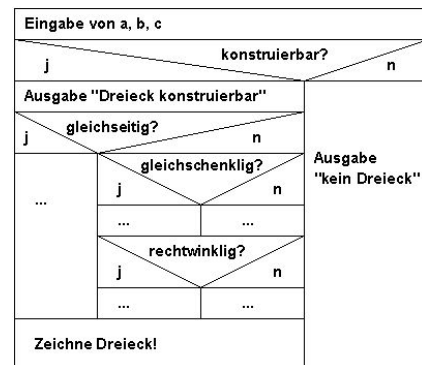
2. Die Seitenlängen eines Dreiecks sind einzulesen. Stelle fest,
 - ob mit diesen Angaben ein Dreieck konstruiert werden kann ² und wenn ja,
 - ob es sich um ein gleichseitiges, ein gleichschenkliges oder ein ungleichschenkliges Dreieck handelt und
 - ob das Dreieck einen rechten Winkel besitzt.

- a) Gegeben ist ein Struktogramm in Grobform. Formuliere die abzufragenden Bedingungen verbal bzw. durch mathematische Aussagen!

- b) In den Abfragen sind offensichtlich mehrere Einzelbedingungen miteinander zu verknüpfen.

Informiere dich über die Verwendung der Booleschen Operatoren AND sowie OR! Benutze die LAZARUS- Hilfe.

- c) Erstelle mit Lazarus eine geeignete Oberfläche für dein Programm!



3. Zahlenraten gegen den Computer

Entwirf und implementiere ein Programm, mit dem ein Benutzer gegen den Computer spielen kann. Dazu „denkt“ sich der Computer eine Zahl (zum Beispiel zwischen 1 und 100), die der Benutzer erraten soll. Er hat mehrere Versuche, eine Zahl einzugeben, wobei der Computer jedes Mal antwortet, ob die Zahl größer oder kleiner als seine „Gedachte“ ist. Hat der Benutzer die Zahl richtig geraten, so gibt der Computer die entsprechende Antwort.

```

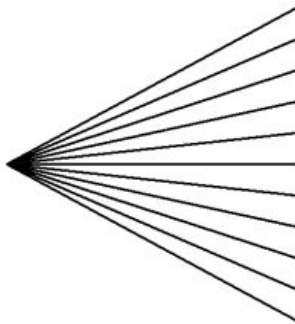
1  var zufallszahl1 : real;
2      zufallszahl2 : integer;
3  begin
4      // Wähle die richtige random-Anweisung
5      zufallszahl1 := random();           // 0 <= zufallszahl < 1
6      zufallszahl2 := random(100);       // 0, 1, 2, ..., 98, 99
7      . . .

```

²Dreiecksungleichung

5.4. Strukturen zur Wiederholung

Zählschleife



```

1 BEGIN
2   Canvas.MoveTo(40,225);
3   Canvas.LineTo(410,25);
4   Canvas.MoveTo(40,225);
5   Canvas.LineTo(410,65);
6   Canvas.MoveTo(40,225);
7   Canvas.LineTo(410,105);
8   .

```

Es werden stets die gleichen Anweisungen mit teilweise anderen Argumenten ausgeführt!
 -> Zahlenfolge: $25 + 40 \cdot n$

```

1 Canvas.MoveTo(40,225);
2 Canvas.LineTo(410,25+40*n);

```

Diese beiden Anweisungen müssen genau 11-mal ausgeführt werden, beginnend mit $n = 0$.

FOR-Statement:

```

FOR variable:= expression TO expression DO
    statement.

```

Die Sequenz zum Zeichnen der Strahlen würde jetzt so lauten:

```

1 VAR n : integer;
2 BEGIN
3   FOR n:=0 TO 10 DO BEGIN
4     Canvas.MoveTo(40,225);
5     Canvas.LineTo(410,25+40*n);
6   END
7 END;

```

FÜR (variable;start;ende)

anweisung;

Eine Zählschleife wird verwendet, wenn die Anzahl der Schleifendurchläufe vorher bekannt sind! Dabei wird die Zählvariable jeweils um 1 herauf bzw. herab gezählt.

Sie darf im Schleifenkörper verwendet, aber **nicht** verändert werden!

- variable muss eine einfache, abzählbare lokale Variable sein.
- start und ende sind Ausdrücke, die zur Variablen zuweisungskompatibel sind.
- start und ende (Abbruch der Schleife) wird nur einmal am Anfang der Schleife ausgewertet.
- Sollen mehr als nur eine Anweisung im Schleifenkörper ausgeführt werden, so müssen diese in **begin** und **end** eingeschlossen werden (siehe 5.2).

Merke

Zusammenfassung

- Die Anzahl der Schleifendurchläufe muss vorher bekannt sein.
- Als Zähler sind nur abzählbare Datentypen erlaubt.
- Es kann immer nur in Einer-Schritten gezählt werden.

Achtung — Semikolonfalle

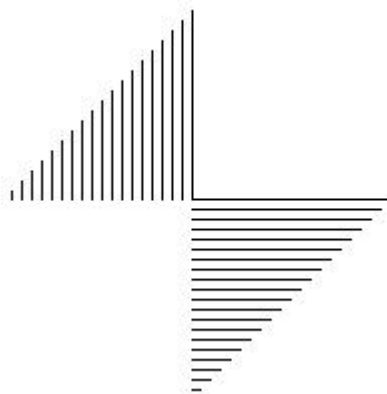
```

.
.
FOR i:=0 TO 10 DO;
    Mem.Lines.Add(IntToStr(i));
.
.

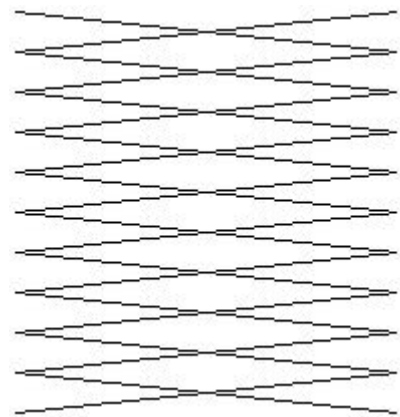
```

● Aufgabe 4 S

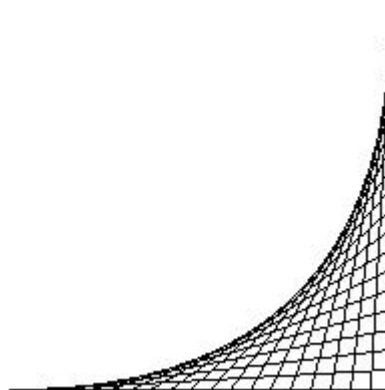
schreibe ein Programm in Objekt Pascal, welches die untenstehenden Figuren in eine Paintbox zeichnen kann. (Verwende für jede Figur einen eigenen Button!)



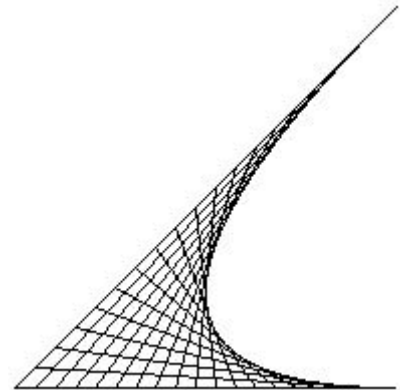
(a) Bild 1



(b) Bild 2



(c) Bild 3



(d) Bild 4

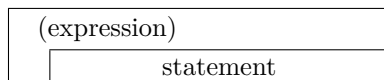
Beachte: Es genügt hier jeweils *eine* Schleife mit genau *einer* Variablen.

Wiederholungen in Abhängigkeit von Bedingungen

... mit Eintrittsbedingung

WHILE-Statement =

```
WHILE expression DO
  statement.
```



Abweisende Schleife

Je nach Bedingung kann der Schleifenkörper übersprungen werden

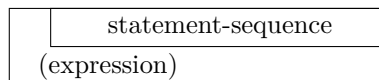
!!! Auch hier *Semikolonfalle* !!!

```
a := 100;
WHILE a > 0 DO BEGIN
  Memo1.Lines.Add(IntToStr(a))
  ;
  a := a - 10;
END;
```

... mit Austrittsbedingung

REPEAT-Statement =

```
REPEAT
  statement-sequence
UNTIL expression.
```



Ausführende Schleife

Der Schleifenkörper wird mindestens einmal ausgeführt

```
a:=100;
REPEAT
  Memo1.Lines.Add(IntToStr(a));
  a := a - 10;
UNTIL a = 0;
```

Beide Schleifen zählen den Inhalt der Ganzzahlvariablen **a** mit 100 beginnend in 10 Schritten bis 0 herunter und geben den jeweiligen Wert von **a** in einem **Memo** aus. Beachte die Unterschiede bei den Anweisungen!

Fließkommaphänomen

```
VAR zahl : real;
BEGIN
  zahl := 0;
  WHILE zahl <> 1 DO BEGIN
    Memo1.Lines.Add(FloatToStr(zahl));
    zahl := zahl + 0.1;
  END;
END;
```

Was leistet das obenstehende Codefragment?

Erstelle ein Programm, welches diese Ausgaben realisiert. Notiere deine Erkenntnisse.

Regula Falsi

Mit dem Sekantenverfahren ist es möglich, die Nullstellen von nichtlinearen Funktionen näherungsweise zu bestimmen.

Gesucht ist die Nullstelle einer Funktion $f(x)$.

Schreibe ein Programm, welches nach dem Sekantenverfahren die Nullstellen der rationalen Funktion

$$f(x) = 0,4x^3 + 1,3x^2 + 0,9x - 0,5$$

berechnet.

Bedenke das Fließkommaphänomen, wenn du die Abbruchbedingung formulierst!

(Hilfe findest du in deinen Aufzeichnungen zur Tabellenkalkulation)



Abbildung 5.1.: Regula Falsi

● Aufgabe 5 F

adengrafiken lassen sich mit WHILE- bzw. REPEAT-Anweisungen fast noch »leichter« zeichnen, wie folgender Quelltext zum Zeichnen der Figur 5 zeigt.

```
VAR x,y : integer;
BEGIN
  my:=PaBox1.Height;
  x:=5; y:=my-5;
  WHILE y>0 DO BEGIN
    WITH PaBox1.Canvas DO
      BEGIN
        MoveTo(x,(my-10)DIV 2);
        LineTo(x,y);
      END;
    x:=x+10;
    y:=y-10;
  END
END;
```

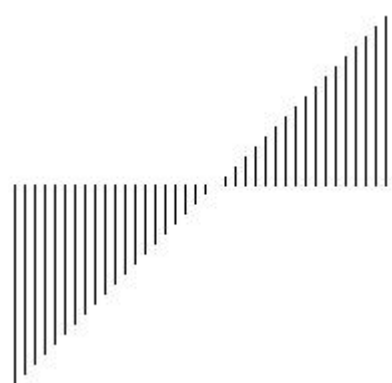


Abbildung 5.2.: Figur 5

»Erfinde« weitere Fadengrafiken!

5.5. Übungen

Bestimme anhand der Aufgabe, ob eine FOR-, eine WHILE- oder eine REPEAT-Schleife genommen werden sollte.

Setze deine Überlegungen in je ein *Struktogramm* **und** ein *Programm* um.

1. Ein Kapital wird zu $p\%$ pro anno auf Zinseszins angelegt, und zwar solange, bis es sich verdoppelt hat.
Schreibe ein Programm, das es dir ermöglicht, den Zusammenhang zwischen Verdopplungszeit und Zinssatz herauszubekommen.
2. Entwickle ein Programm, das das Folgende leistet.
Der Benutzer gibt einige ganze Zahlen ein, der Computer nennt nach jeder Eingabe die aktuell größte und kleinste der bereits eingegebenen Zahlen.
3. Zu gegebenen positiven Zahlen soll der Computer das arithmetische Mittel berechnen.
(Verwende als Abbruchbedingung die Eingabe einer negativen Zahl und achte darauf, dass diese nicht in die Berechnung des Mittelwertes eingeht.)
4. Zu einer natürlichen Zahl n sollen alle Teiler ermittelt werden. Lass dazu zunächst eine Zählvariable von 1 bis n laufen. Überlege dann, ob es auch weniger aufwendig geht; bedenke, dass Teiler stets in Paaren (Teiler, Gegenteiler) auftreten. Benutze zum Bestimmen der Teilbarkeit die Restdivision MOD; a teilt b genau dann, wenn $b \text{ MOD } a$ gleich 0 ist.

5.6. Algorithmusbegriff II

DEFINITION 1

„Unter einem Algorithmus versteht man eine Verarbeitungsvorschrift, die so präzise formuliert ist, dass sie von einem mechanisch oder elektronisch arbeitenden Gerät durchgeführt werden kann.“

(Duden „Informatik“ - Mannheim; Leipzig; Wien; Zürich: Dudenverlag, 1993)

DEFINITION 2

„Ein Algorithmus ist ein Verfahren mit einer präzisen endlichen Beschreibung unter Verwendung effektiver Verarbeitungsschritte.“

(Broy; Informatik Bd. 1 - Berlin; Heidelberg; New York: Springerverlag, 1998)

Es gibt noch eine Reihe weiterer Definitionen von Algorithmen, so auch auch die von uns am Anfang des Kapitels formulierte. Wesentlich sind die Eigenschaften eines Algorithmus, welche in den Definitionen in unterschiedlicher Weise wiedergegeben werden.

Allgemeinheit Typischerweise lösen Algorithmen nicht nur eine Aufgabe, sondern eine Klasse von Aufgaben. Die jeweils betrachtete Aufgabe aus dieser Klasse wird durch Parameter bestimmt.

→ schriftliche Addition der natürlichen Zahlen - alle natürlichen Zahlen lassen sich addieren

Determiniertheit Die Formulierung ist korrekt. Bei gleichen Eingabewerten ergeben sich immer dieselben Ausgabewerte.

→ schriftliche Addition nat. Zahlen: $2 + 2$ ist immer 4

Finitheit Die Verarbeitungsvorschrift lässt sich durch einen endlich langen Text eindeutig formulieren

→ schriftliche Addition nat. Zahlen: auch sehr, sehr große Zahlen lassen sich addieren

Terminierung Für jede Eingabe gibt es nach endlich vielen Schritten, also nach endlich langer Zeit ein Ergebnis. Er hält an. Paradox: Dass es kein Ergebnis gibt, ist auch ein Ergebnis.

→ schriftliche Addition nat. Zahlen: wir erkennen, dass wir für unendlich große Zahlen zu keinem Ende kommen würden.

Determinierung Zu jedem Zeitpunkt der Ausführung gibt es höchstens eine Möglichkeit diese Vorschrift fortzusetzen.

→ schriftl. Addition nat. Zahlen: wir wissen ganz genau, worin der nächste durchzuführende Schritt besteht.

Woher kommt der Begriff „Algorithmus“?

(a) aus dem griechische Begriff arithmos (= Zahl)

(b) von dem arabischen Namen *Al-Chwarismi*

Mohammed Ibn Musa Al-Chwarismi (Mathematiker, Astronom und Geograph am Hof des Kalifen von Bagdad - um 800) beschrieb das indische Dezimalsystem (mit den Ziffern 1 bis 9, besonders der 0) und beschrieb Rechenverfahren in diesem Zahlensystem. Diese Arbeiten wurden später ins Lateinische übersetzt. Vielleicht daher ...

6

Lerneinheit Sechs

Datenstrukturen

6.1. Zeichenketten

Wie bereits im Abschnitt 4.3 beschrieben, wird der Datentyp `string` zum Speichern von Zeichenketten, also Worten, Sätzen oder gar vollständigen Absätzen verwendet. Im Gegensatz zu handschriftlichen Texten gehört die Trennstelle zweier Worte zum Zeichensatz und stellt selbst ein Zeichen dar.

```
var wort: string;
begin
  wort := 'Carl_Zeiss_ist_der_Namensgeber_unserer_Schule!';
  ...
end;
```

Die interne Darstellung der Zeichenkette `wort` wird von *Free Pascal*, solange die Länge nicht 255 Zeichen überschreitet wie folgt realisiert¹:

Tabelle 6.1.: Codierung von *wort*

Index	(0)	1	2	3	4	5	6	7		43	44	45	46
Zeichen	(.)	C	a	r	l		Z	e	...	u	l	e	!
ANSI	(46)	67	97	114	108	32	90	101		117	108	101	33

Das bedeutet, dass auf die einzelnen Zeichen der Zeichenkette wahlfrei im Bereich von 1 bis zur Länge der Zeichenkette zugegriffen werden kann.

```
var wort      : string;    // Zeichenkette
    zeichen   : char;      // Zeichen
    laenge,
    i         : integer;
begin
  wort := 'LAGER';
  laenge := length(wort);  // Länge der Zeichenkette
                           ermitteln
  for i:=1 to (laenge DIV 2) do begin
    // Tauschen zweier Zeichen in der Zeichenkette
    zeichen := wort[i];
    wort[i] := wort[(laenge+1)-i];
    wort[(laenge+1)-i] := zeichen;
  end;
  TForm1.Memo1.Append(wort); // Ausgabe in ein Memo
end;
```

¹ Beachte die Unterschiede zwischen Short- und AnsiString

Was leistet die unten stehende Methode?

Beschreibe zu jeder Anweisung (ev. Zeile), welche Aufgabe sie besitzt. Notiere gegebenenfalls, auf welche Art und Weise dies geschieht!

```
{H-} // {H} ist ein Compilerschalter
// Was leistet die folgende Methode?
procedure BeispielMethode();
var wort    : string;
    zeichen: char;
    laenge,
    i       : integer;
begin
    wort := 'Carl_Zeiss_ist_der_Namensgeber_unserer_Schule!';
    laenge := integer(wort[0]);
//    laenge := length(wort);
    for i:=1 to laenge do begin
        zeichen := wort[i];
        if (zeichen >= 'a') and (zeichen <= 'z' ) then
            wort[i] := char(ord(zeichen) - 32);
    end;
    TForm1.Memo1.Append(wort);
end;
```

[illegible]

Übungen

1. Entwirf und implementiere ein Programm in *Object Pascal*, welches das erste und letzte Vorkommen eines gewünschten Zeichens in einer beliebigen Zeichenkette findet.
2. Entwirf und implementiere ein Programm in *Object Pascal*, mit dem überprüft werden kann, ob eine eingelesenes Wort oder Satz ein Palindrom ist.
3. Sprachwissenschaftler sind der Ansicht, dass auch nach dem Entfernen von Vokalen der Text noch lesbar bleibt.

Entwickle eine Methode `loescheVokale(satz : String)`, die aus einem String `satz` die Vokale entfernt.

Stimmt es, was die Wissenschaftler sagen?

4. Entwirf und implementiere ein Programm in *Object Pascal*, mit dem Sätze der deutschen Sprache in Morsealphabet übersetzt werden können.
5. Entwickle eine Methode, mit der positive Dezimalzahlen in das Additionssystem der römischen Zahlen umgewandelt werden können und umgekehrt.

6.2. Ein- und mehrdimensionale Reihungen

Schwan, Hecht und Krebs

Wenn unter Freunden Einigkeit nicht herrscht,
 Lauft ihre Sache meist verkehrt,
 Am Ende stehen dann die Dinge schlecht.
 Einmal wollten Schwan und Krebs und Hecht
 Mit einem Leiterwagen eine Fuhre machen
 Und luden auf zusammen alle ihre Sachen;
 Sie zieht mit aller Kraft – die Last ruhrt sich kein Stuck!
 Zwar wurde sich der Wagen leicht bewegen lassen,
 Doch zieht der Hecht hinab ins Wasser,
 Der Schwan will zu den Wolken hin, es kriecht der Krebs zuruck.
 Wer recht hat oder nicht, wer will's von uns entscheiden?
 Bis heute muss die Last jedoch an selber Stelle bleiben.

Iwan A. Krylow (1814)

Ähnlich wie bei einem `ShortString`, der als eine Reihung von Zeichen (`character`) betrachtet werden kann, auf die man entsprechend ihrer Position im `String` einzeln zugreifen kann, gibt es auch eine Datenstruktur, die ähnlich aufgebaut ist und auf deren Zellen, die einen anderen Datentyp als Zeichen repräsentieren, wahlweise zugegriffen werden kann – das `ARRAY`.

Vereinfacht gesagt, ist ein `Array` eine Liste von Werten gleichen Types, auf die wahlfrei mittels eines Index zugegriffen werden kann.

Was hat das jetzt mit dem einfuhrenden Beispiel zu tun?

Um den Wagen bewegen zu konnen, mussen Schwan, Krebs und Hecht eine resultierende Kraft auf den Wagen ausuben. In der Ebene ware es ein Leichtes, durch die graphische Vektoraddition die Resultierende zu ermitteln.

Aber wie sieht es im Raum aus?

Hier hilft die arithmetische Vektoraddition²:

$$\vec{a} + \vec{b} + \vec{c} = (a_1 + b_1 + c_1)\vec{e}_x + (a_2 + b_2 + c_2)\vec{e}_y = \begin{pmatrix} a_1 + b_1 + c_1 \\ a_2 + b_2 + c_2 \end{pmatrix}$$

Fur den dreidimensionalen Raum werden demnach 12 Variablen benotigt:

```
// Deklaration der notwendigen Variablen
var schwanX, schwanY, schwanZ, // die Koordinaten des Schwans
    krebsX, krebsY, krebsZ, // die vom Krebs
    hechtX, hechtY, hechtZ, // die vom Hecht
    resultX, resultY, resultZ : integer; // und fur das Ergebnis
...
// Verwendung der Variablen
// dann ergibt sich die folgende Rechnung:
resultX := schwanX + krebsX + hechtX; // der X-Koordinaten
resultY := schwanY + krebsY + hechtY; // der Y-Koordinaten
resultZ := schwanZ + krebsZ + hechtZ; // der Z-Koordinaten
```

Es geht aber auch kurzer.

² Unter einem Vektor versteht man im engeren Sinn in der analytischen Geometrie ein mathematisches Objekt, das eine Parallelverschiebung in der Ebene oder im Raum beschreibt. Ein Vektor kann durch einen Pfeil, der einen Urbildpunkt mit seinem Bildpunkt verbindet, dargestellt werden. Dabei beschreiben Pfeile, die gleichlang, parallel und gleichorientiert sind, denselben Vektor. In kartesischen Koordinaten werden Vektoren durch Zahlenpaare (in der Ebene) bzw. -tripel (im Raum) dargestellt, die oft untereinander (als „Spaltenvektoren,“) geschrieben werden. (Wikipedia)

Da Schwan, Krebs, Hecht und Resultierende aus drei Komponenten (Koordinaten) gleichen Typs (integer) bestehen, deklarieren wir ein ARRAY und verwenden dies zur Berechnung:

```
// Deklaration der notwendigen Variablen als Feld
var schwan, krebs, hecht, result : ARRAY[0..2] OF integer;
...
// Verwendung der Feld-Variablen
// nun ergibt sich die folgende Rechnung:
for i:=0 to 2 do
    result[i] := schwan[i]+krebs[i]+hecht[i];
```

● Aufgabe 6

Entwirf und implementiere ein Programm in *Object Pascal*, mit dem man die Fabel von Iwan Krylow testen kann.

Folgende Festlegungen sollen dabei gelten:

Schwan Die z-Koordinate ist konstant: z=4; die y-Koordinate ist gleich null und die z-Koordinate ist variabel.

Krebs Die x-Koordinate ist konstant: x=-3; die anderen sind gleich null.

Hecht Die z-Koordinate ist konstant: z=-2; x- und y-Koordinate sind variabel.

In dem oben aufgeführten Beispiel handelt es sich um ein eindimensionales statisches ARRAY.

Eindimensional, weil nur *ein* Index zum wahlweisen Zugriff auf die Elemente der Reihung bzw. Feldes notwendig ist, und

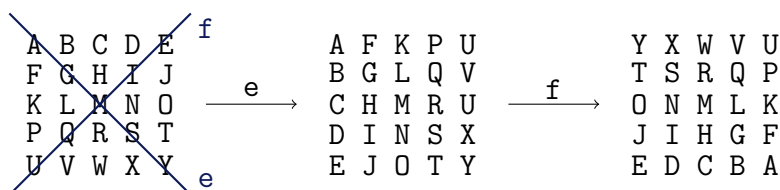
statisch, weil bereits während der Deklaration die Größe der Reihung, besser die Anzahl der Elemente festgelegt wird.

Neben eindimensionalen, können auch mehrdimensionale Array's deklariert werden. Ein typisches Array der Dimension 2 ist das Schachbrett.

```
// Deklarationen
var schachbrett : ARRAY[0..7,0..7] OF tFigur;
    bauer : tFigur;    // im Bsp. nur bauer
...
// Definition und Verwenden
// setzt den Bauern in die 2. Zeile in die 4. Spalte
schachbrett[1,3] := bauer;
// der Bauer schlägt eine andere Figur,
// die rechts vor ihm steht
schachbrett[2,4] := schachbrett[1,3];
```

● Aufgabe 7

In einem Feld (ARRAY [0..4,0..4] of char;) sind die Buchstaben 'A' bis 'Y' zeilenweise einbeschrieben.



Entwirf und implementiere ein Programm in *Object Pascal*, welches die Buchstaben des Feldes nacheinander an den Hauptdiagonalen spiegelt.

Im Allgemeinen werden Variablen bei der Programmerstellung festgelegt, damit entsprechender Speicherplatz im Datensegment für die Werte reserviert werden kann. Bei Variablen von einfachen Datentypen ist es auch sinnvoll, dies zu tun.

Anders jedoch, wenn es sich um zusammengesetzte Datentypen handelt. So bietet **Free Pascal** die Möglichkeit, Arrays zu deklarieren, deren Größe erst während der Laufzeit des Programmes definiert wird und somit eine entsprechende Speicherplatzanforderung auf dem Heap³ (siehe Lerneinheit 4) vom Programm selbst übernommen werden kann.

Deklaration

```
var dynReihung : ARRAY OF integer;
```

Im Gegensatz zu statischen Arrays wird hier im Datensegment kein Speicherplatz für das Array, sondern nur für einen Zeiger (Pointer), der auf den zu einem späteren Zeitpunkt auf dem Heap angeforderten Speicher verweist.

Definieren der Größe der Reihung

```
SetLength(dynReihung, 5);
```

Durch diese Anweisung wird der Speicherplatz auf dem Heap bereitgestellt und das Array kann genauso wie ein statisches Array verwendet werden. Allerdings beginnt der Index *immer* bei Null!

Definition und Verwendung der Elemente der Reihung

```
dynReihung[0] := 5;

for i:=0 to Length(dynReihung)-1 do
  dynReihung[i] := 0;
```

Der Vorteil dynamischer Arrays gegenüber statischen ist nicht nur mit der Möglichkeit begründet, die Größe der Reihung während der Laufzeit⁴ zu definieren. Desweiteren kann genauso die Größe der Reihung *dynamisch* durch Anwenden der Methode `SetLength()` geändert werden.

```
// Erweitern der Reihung auf 15 Elemente
SetLength(dynReihung, 15);
// Löschen der Reihung auf dem Heap
SetLength(dynReihung, 0);
```

Achtung

```
1 var A, B: array of Integer;
2 begin
3   SetLength(A, 1);
4   A[0] := 1;
5   B := A;
6   B[0] := 2;
7 end;
```

³ Der Heap, auch dynamische Speicher ist ein Speicherbereich, aus dem zur Laufzeit eines Programms zusammenhängende Speicherabschnitte angefordert und in beliebiger Reihenfolge wieder freigegeben werden können.

⁴ Z. B. durch Eingabe der Anzahl der gewünschten Elemente per Tastatur

Übungen

1. Fibonacci-Zahlen sind wie folgt definiert:

$$\begin{aligned}f_0 &= 0 \\f_1 &= 1 \\f_i &= f_{i-1} + f_{i-2}\end{aligned}$$

Entwickle ein Programm in *Object Pascal*, welches, nach Eingabe einer Grenze n ein Array mit Fibonacci-Zahlen füllt und anschließend ausgibt.

Überlege, ob du dich für die statische oder dynamische Variante eines Array entscheidest.

2. Bei CD- und MP3-Spielen gibt es häufig eine Random- oder Shuffle-Funktion. Die ist meist so gebaut, dass die Reihenfolge der Stücke zwar zufällig ist, dass aber alle Stücke gespielt werden und keines davon doppelt.

Eine solche Funktion können wir nun auch ganz einfach schreiben:

Wähle in der Liste der Musiktitel immer zwei Einträge zufällig aus und vertauschen sie. Das macht man einige Male (etwa so viele Male, wie Listeneinträge vorhanden sind).

Entwirf und implementiere ein entsprechendes Programm!

3. Erweitern Sie diese Prozedur so, dass auch ein andere Zitate kommen können; welches Zitat genau, soll über eine Nummer eingegeben werden.

Beispiele für Zitate wären:

- Heinrich, mir graut vor Dir!
- Das also war des Pudels Kern!
- Vom Eise befreit sind Strom und Bäche, ...

Entwirf und implementiere ein Programm in *Object Pascal*, das zufällig solche Zitate auf den Bildschirm schreibt.

4. Das Pascal'sche Dreieck ist wie folgt definiert:

Die erste und letzte Zahl einer Zeile ist immer 1. Ab der dritten Zeile ergibt sich die aktuelle Zahl aus der Summe der beiden darüber stehenden Zahlen.

1		1				
1	1	1 1				
1	2	1	1 2 1			
1	3	3	1	1 3 3 1		
1	4	6	4	1	1 4 6 4 1	
1	5	10	10	5	1	1 5 10 10 5 1

Entwirf und implementiere ein Programm in *Object Pascal*, welches nach Eingabe der Dimension n das Pascal'sche Dreieck in der linken oder rechten Form auf dem Bildschirm ausgibt.

5. Das Sieb des Eratosthenes⁵

Man schreibe mit zwei beginnend alle natürlichen Zahlen bis zu einer definierten Obergrenze auf. Dann markiert man die zwei als Primzahl und streicht alle Vielfachen von zwei aus der Liste. Die kleinste nicht markierte und nicht gestrichene Zahl ist die nächste Primzahl. Sie wird markiert und alle Vielfachen aus der Liste gestrichen. Dieser Vorgang wird solange wiederholt, bis es keine Zahlen mehr zu streichen gibt.

⁵ Eratosthenes von Kyrene hat das Verfahren nicht entdeckt, sondern nur die Bezeichnung „Sieb“ für das schon lange vor seiner Zeit bekannte Verfahren eingeführt.

6.3. Verbund

```
TYPE TPerson = RECORD
    Vorname : string;
    Nachname: string;
    Masse   : integer;
    Groesse : real;
END;
```

Ein Verbund fasst mehrere einfache oder zusammengesetzte Variablen zu einer logischen Einheit zusammen. Die Variablen des Verbundes werden Komponenten genannt und dürfen, im Gegensatz zur Reihung, unterschiedlichen Typs sein.

Im Beispiel wurde ein neuer Datentyp **TPerson** definiert, der bestimmte Eigenschaften einer Person kapselt.

```
1 var person1 : TPerson;
2 begin
3     person1.Vorname := 'Max';
4     person1.Nachname:= 'Mustermann';
5     person1.Masse   := 65; // in Kilogramm
6     person1.Groesse := 1.78;
```

```
<record type> ::= record <field list> end
<field list>  ::= <record section> {;<record section>}
<record section> ::= <field identifier> {, <field identifier>} : <type> | <empty>
<field identifier> ::= <identifier>
<identifier> ::= <letter> {<letter> | <digit>}
```

● Aufgabe 8

Entwirf und implementiere ein Programm in *Object Pascal*, das den Namen, das Geschlecht und das Alter mehrerer Personen einlesen, in einer Reihung (array of *tperson*) speichern und die jüngste Person ausgeben kann.

Eine typische Anwendung des Verbundes ist der Datumstyp:

```
1 datum = RECORD
2     tag   : integer;
3     monat: integer;
4     jahr  : integer;
5 END
```

Es ist charakteristisch für einen Verbund, dass alle Kombinationen gültig sind, solange der Wert einer Komponente im Wertebereich des zugehörigen Datentypes liegt. Das bedeutet, dass das Tripel [32;11;2010] ebenso im Wertebereich der Deklaration liegt, wie ein korrektes Datum [21;10;2009]. Es liegt somit in der Verantwortung des Programmierers (Entwicklers), die semantische Bedeutung⁶ der Wertbelegung eines Verbund zu kontrollieren und nur semantisch korrekte Belegungen zuzulassen.

⁶ In der Informationstheorie versteht man unter der Semantik einer Informationsfolge die Bedeutung dieser Informationsfolge.

Übungen

1. Während einer medizinischen Reihenuntersuchung wird die Größe von 150 Neugeborenen untersucht. Nach Eingabe der Daten soll die kleinste und größte Körperlänge ermittelt werden, sowie der Durchschnittswert.
2. Ein gemeiner Bruch besteht aus Zähler und Nenner. Deklariere einen Typ TBruch als entsprechenden Record. Der Benutzer soll Zähler und Nenner des Bruches eingeben und im Anschluss den gekürzten Bruch angezeigt bekommen. Entwickle ein entsprechendes Programm in *Object Pascal*.

7

Lerneinheit Sieben

Arbeiten mit Dateien

7.1. Textdateien

Wiederholung

- Hierarchisches Dateisystem
- Massenspeicher -> Blöcke (Sektoren), sind zu so genannten Clustern zusammengefasst
- Lesen oder Schreiben als Stream (Folge von Bytes),
es wird ein Block vollständig in einen Buffer übertragen, aus dem dann gelesen werden kann

Einführung

Eine Datei ist eine Speicherfläche beliebiger Größe, die auf dem Gerät aus einem oder mehreren Clustern besteht. Jede Datei erhält außerdem eine Beschreibungsstruktur, die die tatsächliche Größe, Referenzen auf die verwendeten Cluster und evtl. weitere Informationen wie Dateityp, Eigentümer, Zugriffsrechte enthalten kann.

Verzeichnisse sind meist speziell gekennzeichnete Dateien, sie enthalten Dateinamen und Referenzen zu den jeweiligen Beschreibungsstrukturen. Sie werden vom System also ähnlich behandelt.

Die erste Beschreibungsstruktur kann dabei das Ausgangsverzeichnis enthalten. Oft ist der erste Block für einen sogenannten Bootblock reserviert, der für das Starten des Systems verwendet wird. Er ist nicht Teil des eigentlichen Dateisystems.

Dateitypen

Typischerweise arbeiten die Programme aller Programmiersprachen mit *logischen* Dateien (Datei-Repräsentanten), die mit einer *wirklichen* Datei verknüpft werden müssen. Für das Verknüpfen einer logischen Programmdatei mit einer realen Datei und deren Handhabung, existieren in Object Pascal verschiedene Konzepte.

Textdatei Ist eine Folge von Zeichen, die in Zeilen angeordnet ist. Die zu verwaltenden Daten werden jeweils als einzelne Textzeilen interpretiert, die durch eine spezielle Zeilenendesequenz (EOLN()) getrennt sind.

Typisierte Datei Eine solche Datei besteht aus einzelnen Datensätzen mit der gleichen Struktur, sprich dem gleichen Datentyp aller enthaltenen Elemente.

Untypisierte Datei Elementare »Low Level« - Dateiverarbeitung. Der Datei wird dabei keine Struktur zugrunde gelegt. Das Schreiben und Lesen erfolgt Blockweise.

● Aufgabe 9 I

informiere dich in der DELPHI-Hilfe über die folgenden Prozeduren und Funktionen. Notiere dir deren Aufgabe und eventuelle Besonderheiten im Kontext von Textdateien.

- AssignFile, CloseFile
- Reset, Rewrite, Append
- IOResult
- EOF
- EOLN
- ReadLn, WriteLn

Zum Verwalten von Dateien gibt es in OBJECT PASCAL weitere Befehle:

- DeleteFile
- RenameFile
- FileGetAttr
- GetDir
- Mkdir
- Rmdir

Zum Umgang mit Textdateien

Beispiel

Erzeuge ein neues Projekt und speichere dieses in einem eigenen Ordner. Behalte dabei (*ausnahmsweise*) die von Delphi vorgeschlagenen Dateinamen bei. Ändere die folgenden Eigenschaften:

- *Form*: Caption → Textdateien
- *Memo*: ScrollBars → ssVertical
- *Memo*: Lines → Inhalt löschen



```

1  procedure TForm1.BtnOeffnenClick(Sender: TObject);
2  var datei : TextFile;           // logischer Name der Datei
3      name  : String;           // Name der physikalischen
        Datei im DS
4      zeile : String;
5  begin
6      name := 'Unit1.dfm';
7      AssignFile(datei, name);    // Verknüpfen der logischen
        Datei, mit
8                                   // der echten Datei des
        Massenspeichers
9      Reset(datei);              // Öffnet die Datei zum Lesen
10     if IOResult=0 then begin    // Hat alles beim Öffnen
        geklappt?

```

```
11     repeat
12         ReadLn(datei, zeile);      // Lies eine Zeile aus der
            Datei
13         Memo1.Lines.Add(zeile);    // Schreibt den Wert von zeile
            in das Memo
14     until (eof(datei));            // Wiederhole bis Dateiende
            erreicht
15     CloseFile(datei);              // Löst die Verknüpfung wieder
16 end;
17 end;
```

Dialog – Komponenten

Delphi stellt Komponenten zur einfachen Benutzung von Dateien, speziell Dateinamen zur Verfügung. (Komponenten-Palette → Reiter: Dialoge)

Wichtigste Methode dieser Dialogkomponenten ist *Execute*. Es ist eine bool'sche Funktion, die das Dialogfenster öffnet, also den Dialog startet und einen Wahrheitswert zurückliefert, welcher Button betätigt wurde (Ok oder Abbrechen).

Verwenden von *TOpenDialog*

Dieses Beispiel zeigt die Verwendung des „Öffnen“-Dialoges in Ergänzung zum obigen Beispiel.

Die Anweisung (Quelltextzeile) `name := 'Unit1.dfm';` kann damit entfallen, wenn im Dialog dieser Dateiname ausgewählt wird!

```
1  procedure TForm1.Btn0effnenClick(Sender: TObject);
2  var name : String;
3  . . .
4  begin
5      if OpenFileDialog1.Execute then begin          // wenn
            erfolgreich
6          name:=OpenDialog1.FileName;
7          AssignFile(datei,name);                    // Verknüpfe
            Datei mit Handle
8      . . .
```

7.2. Komplexe Aufgaben

Kryptographie

Die Datei `osterspaziergang.txt` soll nach dem Cäsar – Verfahren verschlüsselt in ein Memo sowie in eine Datei deines Projektverzeichnisses ausgegeben werden.

(`osterspaziergang.txt` in `austausch\info10_8` auf `\\turing\daten`)

Hinweis: Die Cäsar - Chiffre ist ein Verschlüsselungsverfahren durch monoalphabetische Substitution. Jede Buchstabe des Klartextes wird stets durch den gleichen Buchstaben ersetzt. Konkret wird bei der Cäsar-Chiffre das Alphabet um 3 Zeichen rotiert.

A	→	D	A B C D E F G H ... S T U V W X Y Z
...			D E F G H I J K ... V W X Y Z A B C
Z	→	C	

A $\xrightarrow{\text{integer} = \text{ord}(\text{char})}$ 65 $\xrightarrow{+3}$ 68 $\xrightarrow{\text{char} = \text{chr}(\text{integer})}$ D

Monoalphabetische Verschlüsselungsverfahren sind leicht zu knacken. Ansatzpunkt ist die Buchstabenhäufigkeit in den einzelnen Sprachen.

E: 17,40% N: 9,78% ... P: 0,79% V: 0,67% ... X: 0,03% Q: 0,02% ¹

Kryptoanalyse

Entwirf und implementiere ein Programm in Object Pascal, welches das folgende leistet:

In einem von der Festplatte eingelesene Geheimtext (z.B. Osterspaziergang) sollen die *absoluten Häufigkeiten* der Buchstaben im Text ermittelt, in einer Reihung gespeichert und in einer geeigneten Form auf dem Bildschirm ausgegeben werden.

Entwerfen bedeutet:

- Zeichne für den wesentlichen Algorithmus ein Struktogramm!
- Welche Datenstruktur ermöglicht eine *Abbildung* des Alphabetes auf eine *Liste* von Zahlen?

Hinweis: Groß-/Kleinschreibung werden nicht unterschieden. Es kann jeder Buchstabe umgewandelt werden (`uppercase` bzw. `upcase`).

Erweiterung: Ermittle die absoluten Häufigkeiten von Binomen und Trinomen.

Codes fester Länge

Eine Folge von Informationen wird als Bitmuster fester Länge codiert. Eine wichtige Größe zur Beschreibung der Fehlersicherheit ist der Hammingabstand. Er wird als der kleinste Abstand zwischen allen Codewörtern definiert

Beispiel: Abstand zwischen Codewörtern : 00000, 01101, 01110

0 0 0 0 0	0 0 0 0 0	0 1 1 0 1
0 1 1 0 1	0 1 1 1 0	0 1 1 1 0
hamm = 3	hamm = 3	hamm = 2

$\text{hamm}_{\text{dist}} = 2$ Hammingabstand der Codierung $\Leftarrow$ kleinster Abstand zweier Codewörter

Entwirf und implementiere ein Programm in Object Pascal, welches die in einer Textdatei abgelegten Codewörter einliest und aus diesen den Hammingabstand der Codierung ermittelt.

¹<http://de.wikipedia.org/wiki/Buchstabenhäufigkeit> (Dez. 2006)

```
\\turing\daten .. austausch\info10_08 – Textdateien mit Code... beginnend )
```

International Standard Book Number

Seit dem 01.01.2007 wurde der bis dahin verwendete 10-stellige ISBN-Code auf einen mit 13 Stellen erweitert. Die Umstellung erfolgte ganz einfach:

$$\text{ISBN}(13) = 978 + \text{ISBN}(10)$$

Allerdings mussten die Prüfziffern neu berechnet werden.

Entwirf und implementiere ein Programm in Object Pascal, welches dies leistet!

Aufbau der ISBN(10):

GEISLER, F.; GEISLER, S.: *Delphi - gepackt*; mitp-Verlag/Bonn 2002; ISBN 3-8266-0740-6

3	-	8266	-	0740	-	6
Länder- kennung		Verlag		interne Verlags- nummer		Prüfsumme

Prüfsumme =

$$(z_1 + 3 \cdot z_2 + z_3 + 3 \cdot z_4 + z_5 + 3 \cdot z_6 + z_7 + 3 \cdot z_8 + z_9 + 3 \cdot z_{10} + z_{11} + 3 \cdot z_{12}) \bmod 10$$

European Article Number (Europäische Artikelnummer)

Die EAN dient, ähnlich wie die ISBN für Bücher, einer weltweit eindeutigen Kennzeichnung von Produkten. Zum automatischen Erkennen dieser Artikelnummer dient der Barcode².



Abbildung 7.1.: Barcode

Informiere dich über die Darstellung von EAN's als Barcode.

Entwickle und implementiere ein Programm, welches EAN's in Barcode-Darstellung überführt!

² <http://www.ogobin.org/home/algorithms/barcodes.php>

Regula Falsi

Problemstellung

Mit dem Sekantenverfahren ist es möglich, die Nullstellen von nichtlinearen Funktionen näherungsweise zu bestimmen.

Gesucht ist die Nullstelle einer Funktion $f(x)$.

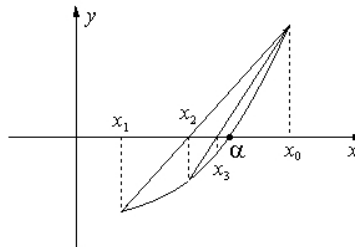
Wählt man zwei beliebige Argumente x_0 und x_1 , ermittelt daraus eine Sekante, so ist die Nullstelle der Sekantengleichung auch Näherungswert der Nullstelle der gesuchten Funktion. Setzt man dieses Verfahren fort, aus x_1 wird x_0 und aus x_2 wird x_1 , so nähert man sich immer mehr an die Nullstelle der gesuchten Funktion an!

Es gilt:

$$m_s = \frac{f(x_2) - f(x_1)}{x_2 - x_1} = \frac{f(x_1) - f(x_0)}{x_1 - x_0}$$

$$f(x_2) = 0$$

$$x_2 = x_1 - f(x_1) \cdot \frac{x_1 - x_0}{f(x_1) - f(x_0)}$$



Entwirf und implementiere ein Programm in Object Pascal, welches das Obenstehende leistet. Verwende die Funktion

$$f(x) = 0,4x^3 + 1,3x^2 + 0,9x - 0,5$$

mit den Startwerten $x_0 = 2$ und $x_1 = 1$.

Zusatz:

Implementiere eine Paintbox, in die der Graph der Funktion in einem gewählten Intervall geplottet werden kann.



Lösung

x0 = 2.0	
x1 = 1.0	
	x2=x1-f(x1)*(x1-x0)/(f(x1)-f(x0))
	Ausgabe(x2)
	x0=x1
	x1=x2
abs(x1-x0)<0,0001	

```

1  function f(x:real):real;           // Funktionsbaustein
2  begin
3      f:=0.4*x*x*x+1.3*x*x+0.9*x-0.5; // Wert des
        Funktionsnamens
4  end;
5
6  procedure TForm1.Button1Click(Sender: TObject);
7  var    x0, x1, x2 : real;
8  const  epsilon=0.0001;             // Genauigkeit
9  begin
10     x0:=StrToFloat(Edit1.Text);     // Startwerte
11     x1:=StrToFloat(Edit2.Text);
12     repeat
13         x2:=x1-f(x1)*(x1-x0)/(f(x1)-f(x0));
14         Memo1.Lines.Add(FloatToStr(x2));
15         x0:=x1;                     // Werte wechseln
16         x1:=x2;
17     until abs(x1-x0)<epsilon;        // Abbruchbedingung
18 end;

```

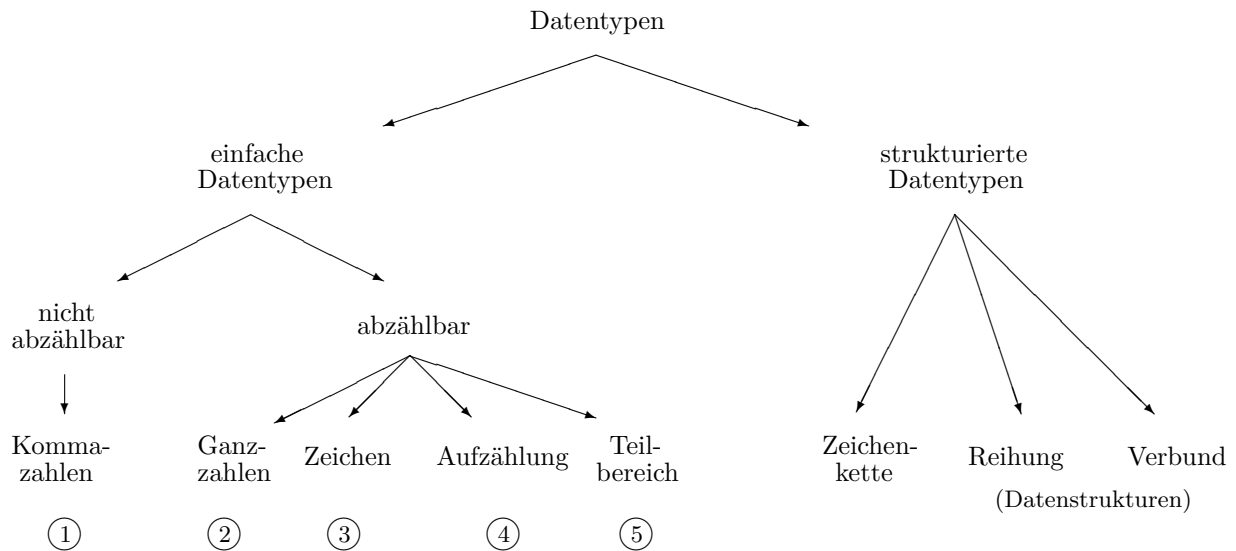
Zusatzaufgabe

Die Funktion $f(x) = 0,4x^3 + 1,3x^2 + 0,9x$ hat nicht mehr nur eine Nullstelle, sondern drei. Von der Wahl der Startwerte hängt es ab, welche Nullstelle vom Programm angenähert wird. Bestimme die drei Nullstellen.

(Kontrollergebnis : 0; -1; -2,25)



Datentypen



1. Eigentlich gebrochene Zahlen in Fließ- bzw. Gleitkommadarstellung

Typ	Bereich	Signifikante Stellen	Größe in Byte
<i>Fundamentale reelle Typen</i>			
Real48	$2.9 \cdot 10^{-39} \dots 1.7 \cdot 10^{38}$	11-12	6
Single	$1.5 \cdot 10^{-45} \dots 3.4 \cdot 10^{38}$	7-8	4
Double	$5.0 \cdot 10^{-324} \dots 1.7 \cdot 10^{308}$	15-16	8
Extended	$3.6 \cdot 10^{-4951} \dots 1.1 \cdot 10^{4932}$	19-20	10
Comp	$-2^{63} + 1 \dots 2^{63} - 1$	19-20	8
Currency	$-922337203685477.5808 \dots 922337203685477.5807$	19-20	8
<i>Generische reelle Typen</i>			
Real	$5.0 \cdot 10^{-324} \dots 1.7 \cdot 10^{308}$	15-16	8

Der generische Typ Real ist in der aktuellen Implementation mit dem Typ Double identisch.

Operationen: + - * / = > < >= <= <>

2. Zahlen in Festkommadarstellung, ohne Nachkommastellen.

Typ	Bereich	Format
<i>Fundamentale Integer-Typen</i>		
Shortint	−128..127	8 Bit, mit Vorzeichen
Smallint	−32.768..32.767	16 Bit, mit Vorzeichen
Longint	−2.147.483.648..2.147.483.647	32 Bit, mit Vorzeichen
Int64	$-2^{63}..2^{63} - 1$	64 Bit, mit Vorzeichen
Byte	0..255	8 Bit, ohne Vorzeichen
Word	0..65.535	16 Bit, ohne Vorzeichen
Longword	0..4.294.967.295	32 Bit, ohne Vorzeichen
<i>Generische Integer-Typen für 32-Bit-Implementationen</i>		
Integer	−2.147.483.648..2.147.483.647	32 Bit, mit Vorzeichen
Cardinal	0..4.294.967.295	32 Bit, ohne Vorzeichen

Operationen: + − * DIV MOD = > < >= <= <>

3. Die fundamentalen Zeichentypen sind *AnsiChar* (8-bit) und *WideChar* (z. Z. 16-bit).

Char, als generischer Datentyp entspricht *AnsiChar* und entspricht dem aktuellen Zeichensatz des jeweiligen Gebietsschemas.

Operationen: CHR()-Nummer in Zeichen, ORD()-Zeichen in Nummer

4. *Aufzählungstypen* definieren eine Menge von Werten mit eindeutiger Reihenfolge, indem einfach die einzelnen Bezeichner dieser Werte aneinander gereiht werden. Die Werte haben *keine* eigene Bedeutung.

Syntax:

```
1 type Typname = (Wert1, ..., Wertn)
```

Beispiel:

```
1 type faecher = (Informatik, Mathematik, Physik, Biologie
, Chemie);
```

5. *Teilbereichstypen* sind definierte Teilmengen anderer *abzählbarer* Datentypen.

Beispiel:

```
1 type kleinBuchstaben = 'a'..'z';
2   grossBuchstaben = 'A'..'Z';
3
4   var kleine : kleinBuchstaben;
5
6   kleine := 'g';      // korrekt
7   kleine := 'A';      // Fehler
```

Zeichenketten

Zeichenketten bestehen aus Folgen von Zeichen (z. B. Buchstaben, Ziffern, Sonder- und Steuerzeichen) und werden durch den Datentyp **String** repräsentiert. Jedes Element eines Strings besitzt den Typ „ANSIChar“ (↑ Tafelwerk).

In *Object Pascal* (32bit-Delphi) gibt es zwei Arten von **Strings**:

- *ShortString* (kurzer String)
- *WideString* (langer String)

Deklaration – Definition – Verwendung

Deklaration Die Verwendung von **String** impliziert einen ANSISString

```
1  var zeichenketteA : String;      // langer String
2      zeichenketteB : String;      // langer String
3      zeichenketteC : String[15];  // kurzer String (15
                                   Zeichen)
```

Definition Es unterscheiden sich zwei Arten der Definition von **String**-Variablen

```
1  zeichenketteA := 'KonstanteZeichenkette';
2  zeichenketteB := '';              // Leerer String
3  SetLength(zeichenketteC,54);      // Leerer String mit
                                   Platz für 54 Zeichen
```

Verwendung von Zeichenketten

String als Datentyp

```
1  zeichenketteA := zeichenketteB + zeichenketteC; } //
                                   Konkatination
2  zeichenketteC := '' + zeichenketteC;
3  zeichenketteA := zeichenketteB + 'Anhängen';
```

Ein **String** als eine Folge von Zeichen, lässt sich auch als ein spezielles Array von Zeichen (ANSIChar) interpretieren. Der Index beginnt grundsätzlich mit 1!

```
1  einChar := zeichenketteA[3];  // Aufpassen — Stringlänge!
2  zeichenketteB[3] := 'x';      // Das dritte Zeichen wird
                                   mit ',x' überschrieben
```

Wichtige Funktionen und Prozeduren für **String**'s¹

```
1  function uppercase (zk : String) : string; // Wandelt um
                                   in Großbuchstaben
2  function length (zk : String) : integer;   // liefert die
                                   Länge eines Strings
3  function Pos(Substr: string; S: string): Integer;
4  function Copy(S; Index, Count: Integer): string;
5  procedure Insert(Source: string; var S: string; Index:
                                   Integer);
6  procedure Delete(var S: string; Index, Count: Integer);
7  function Trim(const S: string): string;
```

¹siehe Delphi-Hilfe

StrToInt()	konvertiert eine Zeichenkette in eine Integer-Zahl <code>iZahl:=StrToInt ('1000');</code>
IntToStr()	konvertiert eine Integer-Zahl in eine Zeichenkette <code>sKette:=IntToStr(1000);</code>
FloatToStr()	konvertiert eine Gleitkommazahl in eine Zeichenkette <code>sKette:=FloatToStr(123.78);</code>
StrToFloat()	konvertiert eine Zeichenkette in eine Gleitkommazahl <code>rZahl:=StrToFloat('123.78');</code>

Beispiele

Zeichenweise umwandeln in Großbuchstaben

```

1  var laenge : integer;
2      zkette : String;
3  begin
4      zkette := 'InFoRmAtIk';
5      laenge := length(zkette);
6      for i:=1 to laenge do           // für jedes Zeichen
7          zkette[i] := upcase(zkette[i]);

```

Vergleichen zweier Zeichenketten

```

1  var text1, text2: String;
2      ergebnis : integer;
3  begin
4      text1 := 'abcde';
5      text2 := 'acdef';
6      ergebnis := AnsiCompareText(text1, text2);
7      if ergebnis < 0 then
8          // text1 ist kleiner als text2
9      else if res>0 then
10         // text1 ist größer als text2
11      else if res=0 then
12         // Strings sind gleich

```

Zeichen in einem String finden

```

1  position := pos('Inf', 'Grundkurs_Informatik'); // position
    ist nun 11

```

Teilstring kopieren

```

1  var text, wort: String;
2  begin
3      text := 'Das_ist_wirklich_einfach!';
4      wort := Copy(text, 8, 8); // wort enthält nun "wirklich"

```

Zeichen in einem String ersetzen

```

1  var text: String;
2  begin
3      text := 'Das_ist_wirklich_einfach!';
4      text := StringReplace(text, 'einfach', 'super', []);

```

Zeichen aus einem String löschen

```

1  var text: String;
2  begin
3      text := 'Das_ist_wirklich_einfach!';
4      Delete(text, 8, 9); // text enthält nun "Das ist einfach
    !"

```

IV

Anhang

Arten von Programmiersprachen

Die einfachste aller Programmiersprachen ist die so genannte Maschinensprache. Als Maschinensprache oder auch Maschinencode versteht man ein System von Instruktionen und Daten, die der Prozessor eines Computers direkt ausführen kann. Ein Wort der Maschinensprache besteht aus einer Sequenz von Bits.

Eine erste Erleichterung brachte die Darstellung des Maschinencodes als Hexadezimalcodes.

Assemblersprachen formulieren die Prozessorbefehle des Maschinencodes als Mnemonics in einer einfachen, relativ leicht lesbaren Syntax. Diese werden vom Assembler in den Maschinencode übersetzt.

10111000	B8	ORG 100H
00000111	07	MOV AX,CS
00001100	0C	MOV DS,AX
11001101	CD	MOV AH,09H
00100001	21	MOV DX,Meldung
10110100	B4	INT 21H
01001100	4C	MOV AX,4C00H
11001101	CD	INT 21H
00100001	21	Meldung: DB "Hello World"
		DB "\$"

Höhere Programmiersprachen formulieren Computerprogramme in einer aus der Sicht eines Computers relativ abstrakten Hochsprache, welche von einer speziellen Software, dem so genannten Compiler in Maschinencode übersetzt wird.

```
public class Square {  
  
    public static void main(String[] args) {  
        for(int i = 0; i < 10 ; ++i)  
            System.out.println(i*i);  
    }  
  
}
```

Eine weitere Abstraktionsstufe stellen unter anderem Datenbanksprachen wie zum Beispiel die Structured Query Language dar.

```
SELECT schueler.Name  
FROM schueler, ausleihen  
WHERE (ausleihen.SNr = schueler.SNr)  
GROUP BY schueler.Name  
HAVING COUNT(schueler.name) > 1  
LIMIT 0 , 30
```




V.1. Verzweigte Strukturen

1. Bei den Tarifverhandlungen des Überbau-Gewerbes einigte man sich darauf, die Gehälter um 3,5%, mindestens jedoch um 100 € zu erhöhen.
Entwirf (Struktogramm) und implementiere ein Programm in *Object Pascal*, das nach Eingabe des alten Gehaltes das zugehörige neue Gehalt errechnet und ausgibt.
2. Gegeben sind zwei Punkte $P = (x_1, y_1)$ und $Q = (x_2, y_2)$.
Gesucht ist die Gleichung $y = m \cdot x + n$ der Verbindungsgeraden. Berücksichtige alle Möglichkeiten der Lage der Punkte bzw. der Geraden.
Entwirf und implementiere ein Programm in *Object Pascal*, das den Anstieg m sowie den y-Achsenabschnitt n berechnet und die Geradengleichung ausgibt.
3. Der Body-Mass-Index (BMI) wird nach der Formel

$$\frac{\text{Körpergewicht (in kg)}}{\text{Körpergröße}^2(\text{in m})}$$

errechnet. Ein BMI unter 18 bedeutet Untergewicht, zwischen 18 und 25 Normalgewicht, zwischen 25 und 30 leichtes Übergewicht und ab 30 Übergewicht.
Entwirf und implementiere ein Programm in *Object Pascal*, das den BMI einer Person errechnet.

V.2. Wiederholende Strukturen

1. Entwirf und implementiere eine Konsolenanwendung in *Object Pascal*, die jeweils das Muster in den dargestellten Formen aus n Zeilen erzeugen kann (n ist eine positive ganze Zahl).

$n = 7$	$n = 6$	$n = 7$	$n = 7$
X X X X X X X	X X X X X X X X X X	X	X X X X X X X
X X X X X X	X X X X X X X X X	X X X	X X X X X
X X X X X	X X X X X X X	X X X X X	X X X
X X X X	X X X X X	X X X X X X X	X
X X X	X X X	X X X X X	X X X
X X	X	X X X	X X X X X
X		X	X X X X X X X

2. Zu gegebenen positiven Zahlen soll das arithmetische Mittel berechnet und ausgegeben werden.
Die Zahlen sollen zeilenweise in ein Memo eingegeben und mit einer negativen Zahl abgeschlossen werden. Nach Betätigen eines Buttons erfolgt die Auswertung.
Entwirf und implementiere ein entsprechendes Programm in *Object Pascal*.
3. Die Länge eines Schiffes (in Meter) ist eine ganze Zahl, der Kapitän hat Söhne und Töchter, und er ist älter als die Anzahl seiner Kinder, aber noch nicht 100 Jahre alt.
Wie alt ist der Kapitän, wie viele Kinder hat er und wie lang ist sein Schiff, wenn das Produkt dieser drei Zahlen 32118 beträgt?

4. Ein vorgegebener Geldbetrag soll in Münzgeld gewechselt werden. Ziel ist es dabei so wenig wie möglich Münzen zu verwenden. Entwirf und implementiere ein Programm in *Object Pascal*.
5. Entwirf und implementiere ein Programm in *Object Pascal*, das alle Teiler einer natürlichen Zahl ermittelt und in einem Memo ausgibt.
6. *Pythagoreische Tripel*
Darunter versteht man natürlichen Zahlen, die als Längen der Seiten eines rechtwinkligen Dreiecks vorkommen können.

$$x^2 + y^2 = z^2$$

Entwirf und implementiere ein Programm in *Object Pascal*, das alle Tripel bis zu einer vorgegebenen Obergrenze für x , y bzw. z ermittelt.

7. Entwirf und implementiere ein Programm in *Object Pascal*, mit dem Konvertierungen zwischen Zahlen im Binär-, Dezimal- und Hexadezimalsystem durchgeführt werden können.
8. Eine Zahlenfolge (a_n) wird nach folgender Bildungsvorschrift aufgestellt:

$$\begin{aligned} \text{Ist } a_n \text{ gerade, dann gilt } a_{n+1} &= \frac{a_n}{2}, \\ \text{Ist } a_n \text{ ungerade, dann gilt } a_{n+1} &= 3a_n + 1. \end{aligned}$$

Entwirf und implementiere ein Programm in *Object Pascal*, das solche Zahlenfolgen mit unterschiedlichen Startwerten ermittelt und ausgibt.

Aufpassen!

9. Gesucht sind alle natürlichen Zahlen mit der Eigenschaft, dass sie auf ihre Quadratwurzel enden.

$$\sqrt{5776} = 76$$

10. Viele Algorithmen ziehen zur Lösung eines Problems Zufallszahlen zu Rate. Um solche so genannten randomisierten Algorithmen implementieren zu können, ist es notwendig, auf einem Computer Zufallszahlen erzeugen zu können. Dies wirft die Frage auf: Kann man Zufall programmieren? Nein. Da ein Verfahren zur Erzeugung von Zufallszahlen deterministisch sein muss, ist eine durch dieses Verfahren erzeugte Folge von Zahlen immer reproduzierbar, vorausgesetzt, die zu Beginn des Verfahrens geltenden Anfangsbedingungen haben sich nicht verändert. Trotzdem gibt es Algorithmen, mit deren Hilfe eine Folge von Zahlen erzeugt werden kann, die statistisch betrachtet einer Folge von Zufallszahlen ähneln. Daher sprechen wir in diesem Zusammenhang auch immer von Pseudozufallszahlen und bezeichnen ein solches Verfahren als Pseudozufallszahlengenerator. In Delphi berechnet die Funktion `random` eine Pseudozufallszahl vom Typ `Real` im Intervall $[0, 1)$. Vor der Berechnung von Zufallszahlen sollte der Zufallszahlengenerator einmalig durch den Aufruf der Prozedur `randomize` initialisiert werden. Entwerfen und implementieren Sie eine Delphi-Anwendung, mit deren Hilfe die Güte der durch die Funktion `random` gelieferten Pseudozufallszahlen besonders anschaulich getestet werden kann.

V.3. Datenstrukturen

1. Beispiel ????
2. Gegeben eine Menge von Städten, mit ihren Koordinaten. Gesucht die beiden Städte, deren Distanz maximal ist.

Tabelle V.1.: *Koordinaten von Städten mit Gymnasien*

Stadt	Längengrad	Breitengrad
Jena	11.5833	50.9333
Apolda	11.5000	51.0167
Hermisdorf	11.4833	52.1833
Weimar	11.3167	50.9833
Gera	12.0833	50.8667
Bad Berka	11.2833	50.9000
Rudolstadt	11.3333	50.7167
Stadtroda	11.7333	50.8500

Tabellenverzeichnis